```
3
4  path_set = {}
5
6  # define decision variable
7  y = {}
8  for i in range(customerNum):
9      var_name = 'y_' + str(i+1)
10     temp_obj = round(data.disMatrix[0][i+1] + data.disMatrix[i+1][0], 1)
11     y[i] = RMP.addVar(lb = 0, ub = 1, obj = temp_obj, vtype = GRB.CONTINUOUS
           , name = var_name)
12     path_set[var_name] = [[0, i + 1, data.customerNum + 1], temp_obj]
13
14 rmp_con = []
15 row_coeff = [1] * customerNum
16 for i in range(customerNum):
17     rmp_con.append(RMP.addConstr(y[i] == 1))
18
19 # export the model
20 RMP.write('RMP_initial.lp')
```

在求解之前，我们将模型导出为.lp 文件。具体模型信息如下：

VRPTW Column Generation small instance

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5
Subject To
 R0:   y_1                                          = 1
 R1:             y_2                                = 1
 R2:                       y_3                      = 1
 R3:                                 y_4            = 1
 R4:                                           y_5  = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
End
```

我们将所有初始可行列对应的具体路径存储到了字典 path_set 中，方便最后提取最终的解及对应的路径。

1. 第 1 次迭代

我们求解 RMP 的线性松弛，然后得到 RMP 的最优解及 5 个约束的对偶变量的取值，具体代码和运行结果如下。由结果可知，初始解质量并不好，路径总长度为 177.3。

Get Duals

```
RMP.optimize()
print('RMP Obj = ', RMP.ObjVal)
for var in RMP.getVars():
    if(var.x > 0):
        print(var.VarName, ' = ', var.x, '\t path :', path_set[var.VarName])

rmp_pi = RMP.getAttr("Pi", RMP.getConstrs())
rmp_pi

[Out]:
RMP Obj =  177.3
y_1  =  1.0       path : [[0, 1, 6], 37.4]
y_2  =  1.0       path : [[0, 2, 6], 41.2]
y_3  =  1.0       path : [[0, 3, 6], 32.2]
y_4  =  1.0       path : [[0, 4, 6], 36.2]
y_5  =  1.0       path : [[0, 5, 6], 30.3]
Duals =  [37.4, 41.2, 32.2, 36.2, 30.3]
```

接下来我们初始化子问题。子问题的模型见 (13.77)~(13.84)。我们调用 Gurobi 建立子问题的模型，并导出为.lp 文件，具体代码如下：

Subproblem

```
# build subproblem
SP = Model('SP')
# decision variables
x = {}
s = {}
# mu = {}
big_M = 1e5
for i in range(data.nodeNum):
    name = 's_' + str(i)
    s[i] = SP.addVar(lb = data.readyTime[i]
                          , ub = data.dueTime[i]
                          , vtype = GRB.CONTINUOUS
```

```
                            , name = name
                            )
    for j in range(data.nodeNum):
        if(i != j):
            name = 'x_' + str(i) + '_' + str(j)
            x[i, j] = SP.addVar(lb = 0
                                , ub = 1
                                , vtype = GRB.BINARY
                                , name = name)

# set objective function of SP
sub_obj = LinExpr(0)
for key in x.keys():
    node_i = key[0]
    node_j = key[1]
    sub_obj.addTerms(round(data.disMatrix[node_i][node_j],1), x[key])
    sub_obj.addTerms(-rmp_pi[node_i], x[key])

SP.setObjective(sub_obj, GRB.MINIMIZE)

# constraints 1
lhs = LinExpr(0)
for key in x.keys():
    node_i = key[0]
    node_j = key[1]
    lhs.addTerms(data.demand[node_i], x[key])
SP.addConstr(lhs <= data.capacity, name = 'cons_1')

# constraints 2
lhs = LinExpr(0)
for key in x.keys():
    if(key[0] == 0):
        lhs.addTerms(1, x[key])
SP.addConstr(lhs == 1, name = 'cons_2')

# constraints 3
for h in range(1, data.nodeNum - 1):
    lhs = LinExpr(0)
    for i in range(data.nodeNum):
        temp_key = (i, h)
        if(temp_key in x.keys()):
            lhs.addTerms(1, x[temp_key])
```

```
    for j in range(data.nodeNum):
        temp_key = (h, j)
        if(temp_key in x.keys()):
            lhs.addTerms(-1, x[temp_key])
    SP.addConstr(lhs == 0, name = 'cons_3' + '_' + str(h))

# constraints 4
lhs = LinExpr(0)
for key in x.keys():
    if(key[1] == data.nodeNum - 1):
        lhs.addTerms(1, x[key])
SP.addConstr(lhs == 1, name = 'cons_4')

# constraints 5
for key in x.keys():
    node_i = key[0]
    node_j = key[1]
    SP.addConstr(s[node_i] + data.disMatrix[node_i][node_j] - s[node_j] -
       big_M + big_M * x[key] <= 0, name = 'cons_5' + '_' + str(key))
SP.write('SP_iter_1.lp')
```

导出的子问题的模型的具体信息如下（约束较多，我们仅展示部分信息）。

initial Subproblem Model

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   - 18.7 x_1_0 - 35.4 x_1_2 - 33.8 x_1_3 - 34.4 x_1_4 - 33.2 x_1_5
   - 18.7 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 - 16.1 x_3_0 - 28.6 x_3_1 - 27.2 x_3_2
   - 30.2 x_3_4 - 31.2 x_3_5 - 16.1 x_3_6 - 18.1 x_4_0 - 33.2 x_4_1
   - 32.6 x_4_2 - 34.2 x_4_3 - 33.2 x_4_5 - 18.1 x_4_6 - 15.2 x_5_0
   - 26.1 x_5_1 - 24.5 x_5_2 - 29.3 x_5_3 - 27.3 x_5_4 - 15.2 x_5_6
   + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4 + 15.1 x_6_5
Subject To
 cons_1: 10 x_1_0 + 10 x_1_2 + 10 x_1_3 + 10 x_1_4 + 10 x_1_5 + 10 x_1_6
   + 30 x_2_0 + 30 x_2_1 + 30 x_2_3 + 30 x_2_4 + 30 x_2_5 + 30 x_2_6
   + 10 x_3_0 + 10 x_3_1 + 10 x_3_2 + 10 x_3_4 + 10 x_3_5 + 10 x_3_6
   + 10 x_4_0 + 10 x_4_1 + 10 x_4_2 + 10 x_4_3 + 10 x_4_5 + 10 x_4_6
   + 10 x_5_0 + 10 x_5_1 + 10 x_5_2 + 10 x_5_3 + 10 x_5_4 + 10 x_5_6
   <= 200
```

```
 cons_2: x_0_1 + x_0_2 + x_0_3 + x_0_4 + x_0_5 + x_0_6 = 1
 cons_3_1: x_0_1 - x_1_0 - x_1_2 - x_1_3 - x_1_4 - x_1_5 - x_1_6 + x_2_1
   + x_3_1 + x_4_1 + x_5_1 + x_6_1 = 0
.....
.....
 cons_4: x_0_6 + x_1_6 + x_2_6 + x_3_6 + x_4_6 + x_5_6 = 1
 cons_5_(0,_1): s_0 + 100000 x_0_1 - s_1 <= 99981.31845830772
.....
 cons_5_(6,_5): - s_5 + s_6 + 100000 x_6_5 <= 99984.86725404958
Bounds
 s_0 <= 1236
 912 <= s_1 <= 967
 825 <= s_2 <= 870
 65 <= s_3 <= 146
 727 <= s_4 <= 782
 15 <= s_5 <= 67
 s_6 <= 1236
Binaries
 x_0_1 x_0_2 x_0_3 x_0_4 x_0_5 x_0_6 x_1_0 x_1_2 x_1_3 x_1_4 x_1_5 x_1_6
 x_2_0 x_2_1 x_2_3 x_2_4 x_2_5 x_2_6 x_3_0 x_3_1 x_3_2 x_3_4 x_3_5 x_3_6
 x_4_0 x_4_1 x_4_2 x_4_3 x_4_5 x_4_6 x_5_0 x_5_1 x_5_2 x_5_3 x_5_4 x_5_6
 x_6_0 x_6_1 x_6_2 x_6_3 x_6_4 x_6_5
End
```

接下来我们求解子问题，并且获取子问题产生的新路径及对应的新列。具体代码如下：

Obtain new column and new path

```
SP.optimize()
print('Reduced Cost:', SP.ObjVal)
'''
    ADD NEW COLUMN
'''

# compute path length
path_length = 0
for key in x.keys():
    node_i = key[0]
    node_j = key[1]
    path_length += x[key].x * data.disMatrix[node_i][node_j]
path_length = round(path_length, 2)
path_length

# creat new column
```

```
col_coef = [0] * data.customerNum
for key in x.keys():
    if(x[key].x > 0):
        node_i = key[0]
        if(node_i > 0 and node_i < data.nodeNum - 1):
            col_coef[node_i - 1] = 1

print('new path length :', path_length)
print('new column :', col_coef)

# extract the corrsponding path for current column
new_path = []
current_node = 0
new_path.append(current_node)
while(current_node != data.nodeNum - 1):
    for key in x.keys():
        if(x[key].x > 0 and key[0] == current_node):
            current_node = key[1]
            new_path.append(current_node)
print('new path :', new_path)
```

求解得到子问题的目标函数（即新列的检验数）和对应的新列及相应的路径，具体代码如下：

New column and new path

```
Reduced Cost: -134.9
new path length : 42.42
new column : [1, 1, 1, 1, 1]
new path : [0, 5, 3, 4, 2, 1, 6]
```

其中，new path length 即为路径 [0, 5, 3, 4, 2, 1, 6] 的总长度。由于 Reduced Cost 为 −134.9，小于 0，因此该列可以被加入 RMP 中。

我们根据产生的新列，更新 RMP，具体代码如下：

Iter 1: Update RMP

```
# Update RMP
rmp_col = Column(col_coef, rmp_con)

cnt = 1
var_name = "cg_" + str(cnt)
RMP.addVar(lb = 0.0, ub = 1, obj = path_length, vtype = GRB.CONTINUOUS, name
        = var_name, column = rmp_col)
```

```
RMP.update()
path_set[var_name] =  new_path

print('current column number :', RMP.NumVars)
# export the model
RMP.write('RMP_iter_1.lp')
RMP.optimize()
print('RMP Obj = ', RMP.ObjVal)
for var in RMP.getVars():
    if(var.x > 0):
        print(var.VarName, ' = ', var.x, '\t path :', path_set[var.VarName])

# get the dual variable of RMP constraints
rmp_pi = RMP.getAttr("Pi", RMP.getConstrs())
print('Duals = ', rmp_pi)
```

导出的更新后的 RMP 如下。其中，cg_1 即为新加入的列对应的决策变量。

Iter 1: Updated RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1
Subject To
 R0:   y_1                                                + cg_1 = 1
 R1:             y_2                                      + cg_1 = 1
 R2:                       y_3                            + cg_1 = 1
 R3:                                 y_4                  + cg_1 = 1
 R4:                                           y_5        + cg_1 = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下。经过一次迭代，RMP 的解由 177.3 降低到 42.42，车辆数也从 5 辆减少为 1 辆，解的提升非常显著。

Iter 1: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_1  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 ------------------------------
Duals =  [37.4, 41.2, 32.2, 36.2, 30.3]
```

2. 第 2 次迭代

由于用列生成算法求解 VRPTW 的过程中，子问题的约束是完全相同的，只是目标函数会有变化。我们根据第 1 次迭代之后的对偶变量，更新 Subproblem 的目标函数，然后再次对其求解，并获得新的列和相应的信息，具体代码如下（获得路径、新列的代码与上面展示的完全相同，这里不再重复展示）。

Iter 2: Update the objective of Subproblem

```
# Update objective of SP
sub_obj = LinExpr(0)
for key in x.keys():
    node_i = key[0]
    node_j = key[1]
    sub_obj.addTerms(data.disMatrix[node_i][node_j], x[key])
    sub_obj.addTerms(-rmp_pi[node_i], x[key])

SP.setObjective(sub_obj, GRB.MINIMIZE)
```

更新后的子问题的目标函数如下：

Iter 2: Objective of Subproblem

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   - 18.7 x_1_0 - 35.4 x_1_2 - 33.8 x_1_3 - 34.4 x_1_4 - 33.2 x_1_5
   - 18.7 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 - 16.1 x_3_0 - 28.6 x_3_1 - 27.2 x_3_2
   - 30.2 x_3_4 - 31.2 x_3_5 - 16.1 x_3_6 - 18.1 x_4_0 - 33.2 x_4_1
   - 32.6 x_4_2 - 34.2 x_4_3 - 33.2 x_4_5 - 18.1 x_4_6 - 15.2 x_5_0
   - 26.1 x_5_1 - 24.5 x_5_2 - 29.3 x_5_3 - 27.3 x_5_4 - 15.2 x_5_6
   + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4 + 15.1 x_6_5
Subject To
.....................................
```

求解子问题，结果如下。我们发现，该子问题的目标函数（即检验数）仍然为 −134.9，路径也和第 1 次迭代产生的路径相同。原因是第 1 次迭代后 RMP 线性松弛问题的对偶变量和初始化步骤中的完全相同（均为 [37.4, 41.2, 32.2, 36.2, 30.3]）。这导致在两次迭代中，子问题的目标函数完全相同，最优解也相同，从而产生了相同的列。

New column and new path

```
Reduced Cost: -134.9
new path length : 42.42
new column : [1, 1, 1, 1, 1]
new path : [0, 5, 3, 4, 2, 1, 6]
```

虽然产生了与之前重复的列，我们仍然可以将其加入 RMP 中，继续更新 RMP 并求解。

Update RMP

```
# Update RMP
rmp_col = Column(col_coef, rmp_con)

cnt = 2
var_name = "cg_" + str(cnt)
RMP.addVar(lb = 0.0, ub = 1, obj = path_length, vtype = GRB.CONTINUOUS, name
        = var_name, column = rmp_col)
RMP.update()
path_set[var_name] =  new_path

print('current column number :', RMP.NumVars)
# export the model
RMP.write('RMP_iter_2.lp')
RMP.optimize()
print('RMP Obj:', RMP.ObjVal)
RMP.optimize()
for var in RMP.getVars():
    if(var.x > 0):
        print(var.VarName, ' = ', var.x, '\t path :', path_set[var.VarName])

# get the dual variable of RMP constraints
rmp_pi = RMP.getAttr("Pi", RMP.getConstrs())
rmp_pi
```

导出更新后的 RMP，模型具体信息如下：

Iter 2: Updated RMP

```
\ Model RMP
```

```
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1 + 42.42
      cg_2
Subject To
 R0:   y_1                                                  + cg_1 +
      cg_2 = 1
 R1:             y_2                                        + cg_1 +
      cg_2 = 1
 R2:                       y_3                              + cg_1 +
      cg_2 = 1
 R3:                                 y_4                    + cg_1 +
      cg_2 = 1
 R4:                                           y_5          + cg_1 +
      cg_2 = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
 cg_2 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下。由结果得知，RMP 的目标值并没有改进，仍然为 42.42，但是对偶变量却变为 [37.4, 41.2, 32.2, 36.2, -104.58]，所以下一步迭代中，子问题的最优解也许会产生变化。

Iter 2: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 ------------------------------
Duals =  [37.4, 41.2, 32.2, 36.2, -104.58]
```

3. 第 3 次迭代

根据第 2 次迭代之后 RMP 的对偶变量，更新子问题的目标函数。更新后子问题的目标函数如下：

Iter 3: Objective of Subproblem

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   - 18.7 x_1_0 - 35.4 x_1_2 - 33.8 x_1_3 - 34.4 x_1_4 - 33.2 x_1_5
   - 18.7 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 - 16.1 x_3_0 - 28.6 x_3_1 - 27.2 x_3_2
   - 30.2 x_3_4 - 31.2 x_3_5 - 16.1 x_3_6 - 18.1 x_4_0 - 33.2 x_4_1
   - 32.6 x_4_2 - 34.2 x_4_3 - 33.2 x_4_5 - 18.1 x_4_6 + 119.68 x_5_0
   + 108.78 x_5_1 + 110.38 x_5_2 + 105.58 x_5_3 + 107.58 x_5_4
   + 119.68 x_5_6 + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4
   + 15.1 x_6_5
Subject To
.....................................
```

求解子问题，结果如下。相较第 2 步迭代，子问题得到的路径有所变化。另外，检验数也变为 −104.6，小于 0，说明该列可以加入 RMP 中。

Iter 3: New column and new path

```
Reduced Cost: -104.6
new path length : 42.41
new column : [1, 1, 1, 1, 0]
new path : [0, 3, 4, 2, 1, 6]
```

我们创建新决策变量 cg_3，并将新列 [1, 1, 1, 1, 0] 加入 RMP 并求解，导出的更新后的 RMP 如下：

Iter 3: Updated RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1 + 42.42
      cg_2 + 42.41 cg_3
Subject To
 R0:   y_1                                                  + cg_1 +
      cg_2 +       cg_3 = 1
 R1:             y_2                                        + cg_1 +
      cg_2 +       cg_3 = 1
 R2:                       y_3                              + cg_1 +
      cg_2 +       cg_3 = 1
```

```
 R3:                                    y_4                    + cg_1 +
       cg_2 +        cg_3 = 1
 R4:                                                 y_5        + cg_1 +
       cg_2                = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
 cg_2 <= 1
 cg_3 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下。虽然 RMP 的目标值依旧保持为 42.42，没有任何改进，但是对偶变量的值又一次发生了变化。这说明，下一步迭代中，Subproblem 很有可能再次产生新的不同的列。

Iter 3: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 ------------------------------
Duals =  [37.4, 41.2, 32.2, -68.39000000000001, 0.010000000000019327]
```

4. 第 4 次迭代

根据第 3 次迭代之后 RMP 的对偶变量，更新子问题的目标函数。更新后子问题的目标函数如下：

Iter 4: Objective of Subproblem

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   - 18.7 x_1_0 - 35.4 x_1_2 - 33.8 x_1_3 - 34.4 x_1_4 - 33.2 x_1_5
   - 18.7 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 - 16.1 x_3_0 - 28.6 x_3_1 - 27.2 x_3_2
   - 30.2 x_3_4 - 31.2 x_3_5 - 16.1 x_3_6 + 86.49 x_4_0 + 71.39 x_4_1
   + 71.99 x_4_2 + 70.39 x_4_3 + 71.39 x_4_5 + 86.49 x_4_6 + 15.09 x_5_0
```

```
  + 4.19 x_5_1 + 5.79 x_5_2 + 0.99 x_5_3 + 2.99 x_5_4 + 15.09 x_5_6
  + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4 + 15.1 x_6_5
Subject To
.....................................
```

求解上述子问题，得到下面的结果。这一次，子问题又产生了一个与之前的列不重复的列，检验数也变为 −69，为负，因此该列也可以加入 RMP。

Iter 4: New column and new path

```
Reduced Cost: -69.01000000000002
new path length : 41.81
new column : [1, 1, 1, 0, 1]
new path : [0, 5, 3, 2, 1, 6]
```

继续更新 RMP 并求解，导出的更新后的 RMP 如下：

Iter 4: Updated RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1 + 42.42
      cg_2 + 42.41 cg_3 + 41.81 cg_4
Subject To
 R0:   y_1                                                    + cg_1 +
      cg_2 +       cg_3 +       cg_4 = 1
 R1:             y_2                                          + cg_1 +
      cg_2 +       cg_3 +       cg_4 = 1
 R2:                       y_3                                + cg_1 +
      cg_2 +       cg_3 +       cg_4 = 1
 R3:                               y_4                        + cg_1 +
      cg_2 +       cg_3            = 1
 R4:                                          y_5             + cg_1 +
      cg_2 +                    cg_4 = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
 cg_2 <= 1
 cg_3 <= 1
```

```
 cg_4 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下。RMP 的松弛问题的目标值还是没有变化，但是对偶变量有所变化。

Iter 4: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 ------------------------------
Duals =  [37.4, 41.2, -36.800000000000004, 0.6099999999999994,
      0.010000000000005116]
```

5. 第 5 次迭代

根据第 4 次迭代之后 RMP 的对偶变量，更新子问题的目标函数。更新后子问题的目标函数如下：

Iter 5: Objective of Subproblem

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   - 18.7 x_1_0 - 35.4 x_1_2 - 33.8 x_1_3 - 34.4 x_1_4 - 33.2 x_1_5
   - 18.7 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 + 52.9 x_3_0 + 40.4 x_3_1 + 41.8 x_3_2
   + 38.8 x_3_4 + 37.8 x_3_5 + 52.9 x_3_6 + 17.49 x_4_0 + 2.39 x_4_1
   + 2.99 x_4_2 + 1.39 x_4_3 + 2.39 x_4_5 + 17.49 x_4_6 + 15.09 x_5_0
   + 4.19 x_5_1 + 5.79 x_5_2 + 0.99 x_5_3 + 2.99 x_5_4 + 15.09 x_5_6
   + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4 + 15.1 x_6_5
Subject To
.....................................
```

求解结果如下。检验数比上一次更靠近 0，但仍然为负，算法需要继续迭代。

Iter 5: New column and new path

```
Reduced Cost: -37.3
new path length : 41.3
new column : [1, 1, 0, 0, 0]
new path : [0, 2, 1, 6]
```

继续更新 RMP 并求解，导出的更新后的 RMP 如下：

Iter 5: Updated RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1
   + 42.42 cg_2 + 42.41 cg_3 + 41.81 cg_4 + 41.3 cg_5
Subject To
 R0: y_1 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5 = 1
 R1: y_2 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5 = 1
 R2: y_3 + cg_1 + cg_2 + cg_3 + cg_4        = 1
 R3: y_4 + cg_1 + cg_2 + cg_3               = 1
 R4: y_5 + cg_1 + cg_2        + cg_4        = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
 cg_2 <= 1
 cg_3 <= 1
 cg_4 <= 1
 cg_5 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下：

Iter 5: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 --------------------------------
Duals =  [0.09999999999999432, 41.2, 0.5, 0.6099999999999994,
      0.010000000000005116]
```

6. 第 6 次迭代

根据第 5 次迭代之后 RMP 的对偶变量，更新子问题的目标函数。更新后子问题的目标函数如下：

Iter 6: Objective of Subproblem

```
\ Model SP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  18.7 x_0_1 + 20.6 x_0_2 + 16.1 x_0_3 + 18.1 x_0_4 + 15.1 x_0_5
   + 18.6 x_1_0 + 1.9 x_1_2 + 3.5 x_1_3 + 2.9 x_1_4 + 4.1 x_1_5
   + 18.6 x_1_6 - 20.6 x_2_0 - 39.2 x_2_1 - 36.2 x_2_3 - 37.6 x_2_4
   - 35.4 x_2_5 - 20.6 x_2_6 + 15.6 x_3_0 + 3.1 x_3_1 + 4.5 x_3_2
   + 1.5 x_3_4 + 0.5 x_3_5 + 15.6 x_3_6 + 17.49 x_4_0 + 2.39 x_4_1
   + 2.99 x_4_2 + 1.39 x_4_3 + 2.39 x_4_5 + 17.49 x_4_6 + 15.09 x_5_0
   + 4.19 x_5_1 + 5.79 x_5_2 + 0.99 x_5_3 + 2.99 x_5_4 + 15.09 x_5_6
   + 18.7 x_6_1 + 20.6 x_6_2 + 16.1 x_6_3 + 18.1 x_6_4 + 15.1 x_6_5
Subject To
.....................................
```

求解结果如下。这一次，检验数仅为 −0.02，已经非常接近 0。

Iter 6: New Column and new path

```
Reduced Cost: -0.020000000000006235
new path length : 42.35
new column : [0, 1, 1, 1, 1]
new path : [0, 5, 3, 4, 2, 6]
```

继续更新 RMP 并求解，导出的更新后的 RMP 如下：

Iter 6: Updated RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1
   + 42.42 cg_2 + 42.41 cg_3 + 41.81 cg_4 + 41.3 cg_5 + 42.35 cg_6
Subject To
 R0: y_1 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5          = 1
 R1: y_2 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5 + cg_6 = 1
 R2: y_3 + cg_1 + cg_2 + cg_3 + cg_4          + cg_6 = 1
 R3: y_4 + cg_1 + cg_2 + cg_3                 + cg_6 = 1
 R4: y_5 + cg_1 + cg_2          + cg_4        + cg_6 = 1
Bounds
 y_1 <= 1
 y_2 <= 1
 y_3 <= 1
```

```
 y_4 <= 1
 y_5 <= 1
 cg_1 <= 1
 cg_2 <= 1
 cg_3 <= 1
 cg_4 <= 1
 cg_5 <= 1
 cg_6 <= 1
End
```

求解更新后的 RMP，得到更新后 RMP 的最优解及每个约束的对偶变量的值，具体如下：

Iter 6: Duals and solution of RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [[0, 5, 3, 4, 2, 1, 6], 42.42]
 ------------------------------
Duals =  [0.09999999999999432, 41.2, 0.5, 0.6099999999999994,
      0.010000000000005116]
```

7. 第 7 次迭代

根据第 6 次迭代之后 RMP 的对偶变量，更新子问题的目标函数，并求解子问题，得到如下的解：

Iter 7: Subproblem Solution

```
Reduced Cost: -0.020000000000006235
new path length : 42.35
new column : [0, 1, 1, 1, 1]
new path : [0, 5, 3, 4, 2, 6]
```

我们发现，检验数非常小，接近 0，并且之后的几次迭代，子问题的目标函数都不再变化。因此，可以认为列生成算法的过程已经结束。我们可以直接求解最终的 RMP，获得相应的解。

8. 最终 RMP 的求解

我们将 RMP 中的所有变量设置成 binary 类型，然后对其求解，代码如下：

Final: Solve the final RMP

```
mip_var = RMP.getVars()
for i in range(RMP.numVars):
```

```
    mip_var[i].setAttr("VType", GRB.BINARY)

# export the model
RMP.write('RMP_Final.lp')
# solve the final RMP
RMP.optimize()
print('RMP Obj = ', RMP.ObjVal)
print(' ----- RMP Optimal Solution ----- ')
for var in RMP.getVars():
    if(var.x > 0):
        print(var.VarName, ' = ', var.x, '\t path :', path_set[var.VarName])
```

最终的 RMP 的模型如下：

Final: Solve the final RMP

```
\ Model RMP
\ LP format - for model browsing. Use MPS format to capture full model
      detail.
Minimize
  37.4 y_1 + 41.2 y_2 + 32.2 y_3 + 36.2 y_4 + 30.3 y_5 + 42.42 cg_1
   + 42.42 cg_2 + 42.41 cg_3 + 41.81 cg_4 + 41.3 cg_5 + 42.35 cg_6
   + 42.35 cg_7
Subject To
 R0: y_1 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5                = 1
 R1: y_2 + cg_1 + cg_2 + cg_3 + cg_4 + cg_5 + cg_6 + cg_7 = 1
 R2: y_3 + cg_1 + cg_2 + cg_3 + cg_4        + cg_6 + cg_7 = 1
 R3: y_4 + cg_1 + cg_2 + cg_3               + cg_6 + cg_7 = 1
 R4: y_5 + cg_1 + cg_2        + cg_4        + cg_6 + cg_7 = 1
Bounds
Binaries
 y_1 y_2 y_3 y_4 y_5 cg_1 cg_2 cg_3 cg_4 cg_5 cg_6 cg_7
End
```

根据上述模型，可知 RMP 中一共有 12 列，分别对应 12 条路径和 12 个决策变量，下面是这些决策变量对应的路径信息。

Paths of final RMP

```
{'y_1': [[0, 1, 6], 37.4],
 'y_2': [[0, 2, 6], 41.2],
 'y_3': [[0, 3, 6], 32.2],
 'y_4': [[0, 4, 6], 36.2],
 'y_5': [[0, 5, 6], 30.3],
```

```
'cg_1': [[0, 5, 3, 4, 2, 1, 6], 42.42],
'cg_2': [[0, 5, 3, 4, 2, 1, 6], 42.42],
'cg_3': [[0, 3, 4, 2, 1, 6], 42.41],
'cg_4': [[0, 5, 3, 2, 1, 6], 41.81],
'cg_5': [[0, 2, 1, 6], 41.3],
'cg_6': [[0, 5, 3, 4, 2, 6], 42.35],
'cg_7': [[0, 5, 3, 4, 2, 6], 42.35]}
```

求解最终 RMP，得到的结果如下，由结果可知，RMP 的目标值为 42.42，所用车辆数为 1，其路径为 [0, 5, 3, 4, 2, 1, 6]。

Final: Final RMP

```
RMP Obj =  42.42
 ----- RMP Optimal Solution -----
cg_2  =  1.0     path : [0, 5, 3, 4, 2, 1, 6]
```

根据之前的介绍，RMP 的松弛问题的最优值（即 z_{RLMP}），加上 5 个子问题（每辆车对应一个子问题）目标函数的值（即 $\tilde{c}_i^*, \forall i = 1, 2, \cdots, 5.$），是原问题的一个下界，即 $\text{LB} = z_{\text{RLMP}} + \sum_{i=1}^{5} \tilde{c}_i^*$。而 RMP 的最优解是原问题的一个上界，即 $\text{UB} = z_{\text{RMP}}$。由求解结果得知，$z_{\text{RLMP}} = 42.42$，$\tilde{c}_i^* = -0.02, \forall i = 1, 2, \cdots, 5$（因为 5 辆车是同质的，它们的子问题实际上是相同的，因此目标函数也相同），$z_{\text{RMP}} = 42.42$，因此，原问题最优值 z^* 的范围为

$$\text{LB} = 42.42 - (5 \times 0.02) \leqslant z^* \leqslant 42.42 = \text{UB}$$

LB 和 UB 之间的 Gap 仅为 $(\text{UB} - \text{LB})/\text{UB} = 0.10/42.42 \approx 0.236\%$，因此该解的质量已经相当高，可以大致认为就是最优解（因为语言本身是有数值精度问题的）。

到这里，我们已经向读者展示了列生成算法求解 VRPTW 的完整过程。可以观察到，在算法迭代过程中，子问题的目标函数是不断变化的，因此每次生成的列一般都不相同，这也就使得在迭代过程中会生成很多不同的列，从而使得 RMP 也在不断变化。

注意： 由于最终 RMP 的模型中只包含了每列访问的客户点，并不包含访问顺序的信息，因此仅仅依靠 RMP 的解，是不能获得具体的路径信息的。为了获得具体的路径信息，我们需要在列生成算法的过程中存储相应的路径信息，这样才能在最后对应地将路径提取出来，这也正是代码中 path_set 的作用。

第14章　动态规划

动态规划（Dynamic Programming，DP）是运筹学中的一种重要的最优化数学方法，主要用来求解多阶段决策问题。20 世纪 50 年代初，美国数学家 R. Bellman 等在研究多阶段决策过程的优化问题时，提出了著名的最优化原理，从而创立了动态规划（Bellman 1952，Bellman 1966）。动态规划算法可以非常高效地求解 SPPRC，从而加快 VRPTW 的求解。同时，也可以用于提高其他子问题为 SPPRC 的相关问题的求解效率。

14.1　动 态 规 划

动态规划分为前向动态规划、后向动态规划和双向动态规划。动态规划的应用非常广泛，包括工程技术、经济、工业生产、军事以及自动化控制等领域，并在最短路问题、Lot-Sizing 问题、背包问题、资金管理问题、资源分配问题和复杂系统可靠性等问题中取得了显著的应用效果。

运用动态规划求解的问题一般具备如下特点：① 无后效性：某个阶段的状态一旦确定后，就不会受该状态之后的决策影响，同时，该阶段之后的决策和状态发展不受该阶段之前的各状态的影响。② 最优子结构：如果某个问题的最优解可以由其子问题的最优解推出，那么该问题就具备最优子结构。简言之，一个最优策略的子策略总是最优的，就叫作最优子结构，也叫作最优化原理。③ 子问题的重叠性：动态规划在实现过程中需要存储各种状态，这些状态会占用计算空间，如果要使得动态规划算法能够比较高效，那么各种状态就要有较高的复用性，以便减少计算空间的占用。但该性质不是必需的，只是在一定程度上决定了算法的效率。

动态规划的求解思路如下。

（1）划分阶段：将问题按照时间或者空间特征分为若干相互联系的阶段，这些阶段需要有序或者可排序，否则不便于动态规划过程的递推。其中描述阶段的变量称为阶段变量，阶段变量一般是离散变量。

（2）状态表示：状态主要用来描述问题各阶段所处的客观情况或自然条件，这些情况或条件具备无后效性，即当前状态不会受决策者后续决策的影响。

（3）状态转移：状态表示确定之后，问题从一个阶段向下一个阶段转移时，伴随着状态的跳转，状态的跳转需要状态转移方程（又叫状态递推方程）来表示，状态转移方程一般需要在划分阶段、状态表示之后，根据状态之间的跳转关系进行决策分析确定。

（4）确定边界：初始状态和最终状态的确定，以限制问题的开始和结束。

14.1.1 动态规划求解最短路问题

给定有向图 $D=(V,A)$，V 是图中的点集，A 是弧集，每个弧段都有一个相应的非负距离（权值），要求从一个初始点 s 开始到另一个终止点 t 之间，找到一条使得距离最短（权值最小）的路，就是最短路问题（Shortest Paths Problem）。下面用一个例子详细解释 DP 的过程。

假设小明要驾车从城市 1 去城市 10，中间有多条路径可以选择，每条路径途经的城市（城市 2~9）和路程长度是不一样的，如图 14.1 所示。其中城市 2、3、4 可以驾车在一天内到达，城市 5、6、7 需要驾车两天才能到达，城市 8、9 需要三天时间到达，到达城市 10 则需要四天时间。要使得总行驶距离最小，小明应该如何规划这四天的行程？

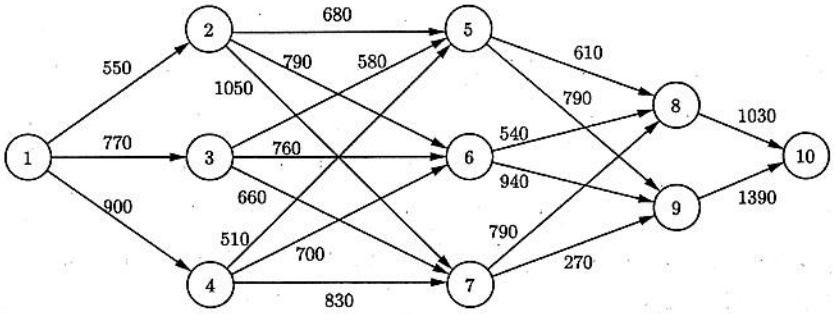


图 14.1 最短路问题：网络结构图

14.1.2 问题建模和求解

接下来我们运用上面理论部分所讲的动态规划的思路来分析这个例题。首先划分阶段，题目中提及各城市到达的时间是不一样的，从一天到四天不等，因此很明显我们可以将时间（天数）作为划分阶段的标准，即将问题划分为四个阶段（$t=\{1,2,3,4\}$）。然后确定状态，我们将状态设定为某一天开始时小明从某个城市出发到达目的地时所驾车驶过的累积路程，我们定义为 $f_t(i), i\in V$。接下来分析状态转移过程，每一天的结束将伴随小明从一个城市去到另一个城市，他所驾车驶过的路程就进行了跳转。我们采用后向的动态规划思路来分析递推公式，假定后一个阶段一定取得最短路，那么当前阶段的状态转移方程可以写为 $f_t(i)=\min\{f_{t+1}(i)+c_{ij}\}$，其中，$j$ 是从 t 到 $t+1$ 阶段所有可以从 i 到达的点，c_{ij} 是点 i 与点 j 之间的距离。最后确定开始和结束条件，因为是后向动态规划，所以开始于第四阶段，结束于第一阶段。

下面我们从第 4 阶段开始分别进行分析动态规划的整个决策求解过程。

第 4 阶段：小明将从城市 8 或者城市 9 出发去往城市 10，因此我们需要分别计算 $f_4(8)$ 和 $f_4(9)$。

$$f_4(8)=\min\{f_5(10)+c_{8,10}\}=\min\{0+1030\}=1030$$

$$f_4(9)=\min\{f_5(10)+c_{9,10}\}=\min\{0+1390\}=1390$$

其中，$f_5(10)$ 已经到达目的地，所以其值为 0。

第 3 阶段：小明将从城市 5 或者城市 6 或者城市 7，去往城市 8 或者城市 9，因此要获得从当前城市到达城市 10 的最短路，需要对城市 5、6、7 分别计算状态转移方程。

$$f_3(5)=\min\begin{cases}f_4(8)+c_{58}=1030+610=1640*\\ f_4(9)+c_{59}=1390+790=2180\end{cases}$$

其中，* 表示 $f_3(5)$ 的最小值，即从城市 5 到城市 10 的最短路径为 $5\to 8\to 10$，将其作为备选路径，以下 * 表示相同含义。

$$f_3(6)=\min\begin{cases}f_4(8)+c_{68}=1030+540=1570*\\ f_4(9)+c_{69}=1390+940=2330\end{cases}$$

$f_3(6)$ 的最小值为 1570，从城市 6 到城市 10 的最短路径为 $6\to 8\to 10$，将其作为途经城市 6 到目的地的备选路径。

$$f_3(7)=\min\begin{cases}f_4(8)+c_{78}=1030+790=1820\\ f_4(9)+c_{79}=1390+270=1660*\end{cases}$$

$f_3(7)$ 的最小值为 1660，从城市 6 到城市 10 的最短路径为 $7\to 9\to 10$，将其作为途经城市 7 到目的地的备选路径。

第 2 阶段：现在我们已经知道了从第 3 阶段的城市 5、6、7 分别到达目的地城市 10 的最优路线，那么继续进行后向动态规划的公式递推，我们将获得从第 2 阶段各城市到达目的地的最短路径。第 2 阶段的城市有 2、3、4，因此需要我们分别计算 $f_2(2)$、$f_2(3)$、$f_2(4)$。

$$f_2(2)=\min\begin{cases}f_3(5)+c_{25}=1640+680=2320*\\ f_3(6)+c_{26}=1570+790=2360\\ f_3(7)+c_{27}=1660+1050=2710\end{cases}$$

$f_2(2)$ 的最小值为 2320，从城市 2 到城市 10 的最短路径可从 $f_3(5)$ 进行追溯，此路径即为 $2\to 5\to 8\to 10$，我们将其作为途经城市 2 到目的地的备选路径。

$$f_2(3)=\min\begin{cases}f_3(5)+c_{35}=1640+580=2220*\\ f_3(6)+c_{36}=1570+760=2330\\ f_3(7)+c_{37}=1660+660=2320\end{cases}$$

$f_2(3)$ 的最小值为 2220，从城市 3 到城市 10 的最短路径为 $3\to 5\to 8\to 10$，将其作为途经城市 3 到目的地的备选路径。

$$f_2(4)=\min\begin{cases}f_3(5)+c_{45}=1640+510=2150*\\ f_3(6)+c_{46}=1570+700=2270\\ f_3(7)+c_{47}=1660+830=2490\end{cases}$$

$f_2(4)$ 的最小值为 2150，从城市 2 到城市 10 的最短路径为 $4 \to 7 \to 8 \to 10$，将其作为途经城市 4 到目的地的备选路径。

第 1 阶段：我们目前已经知道了从城市 2、3、4 到达目的地 10 的最短路径，接下来只需要继续进行后向动态规划过程求解 $f_1(1)$ 就可以获得从 1 到 10 的最短路径。

$$f_1(1) = \min \begin{cases} f_2(2) + c_{12} = 2320 + 550 = 2870* \\ f_2(3) + c_{13} = 2220 + 900 = 3120 \\ f_2(4) + c_{14} = 2150 + 770 = 2920 \end{cases}$$

至此，我们已经探明了从城市 1 到城市 10 的最短距离为 2870，其路径可以通过追溯 $f_2(2)$ 获得，即为 $1 \to 2 \to 5 \to 8 \to 10$。所得最短路如图 14.2 中粗线条所示路径。

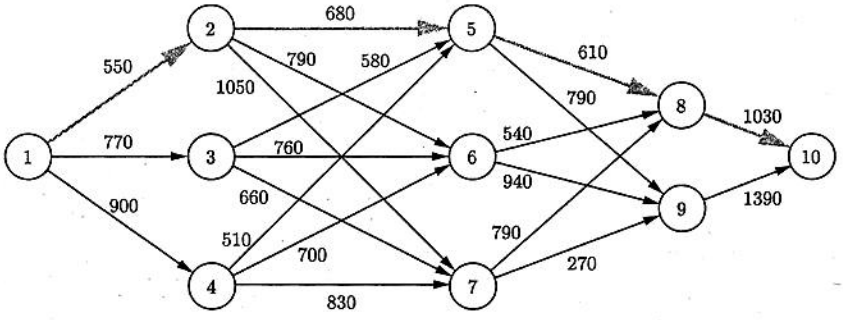


图 14.2 最短路

14.1.3 一个较大规模的例子

上面的例子比较简单，仅用来展示动态规划解决问题的思路，但它可能无法有效地展示运用动态规划的优势所在。其实，对于较大规模的问题，动态规划也能够比较有效地进行求解，比如再举一个稍大规模的例子。

如图 14.3 所示，从点 0 到点 26 的一个分阶段的网络，各相邻两阶段之间互通，任意可通达的两点 (i,j) 之间的距离已知 (c_{ij})，求从初始点 0 到终止点 26 的最短路径。

一种很容易想到的方法就是枚举其中的所有路径，然后对比得出最短路径。但我们注意到，该网络中，从 0 到 26 有 5^5 条可行路径，每条路径的计算都要进行 5 条弧的加和，因此总共需要进行 $5 \times 5^5 = 15625$ 次加和计算，以及 $5^5 - 1 = 3124$ 次比较运算，才能获得最短路径。很明显这种方法不太实用，尤其在实际应用中，网络规模比这个大得多。而如果我们运用上面分析的动态规划的思路进行求解，首先将问题划分为 0~6 共 7 个阶段，$f_6(26)$ 已经到达目的地不需要进行决策，运用后向动态规划从阶段 5 开始，$f_5(i), i \in \{21,22,23,24,25\}$ 不需要进行计算，只需要比较 4 次，阶段 4 至阶段 1 的计算可以用通用递推公式 $f_t(j) = \min\{f_{t+1}(k) + c_{jk}, t \in \{1,2,3,4\}, j \in \{5t-4, 5t-3, 5t-2, 5t-1, 5t\}, k \in \{5t+1, 5t+2, 5t+3, 5t+4, 5t+5\}\}$ 进行计算，对于每个 t, j, k 进行组合，总共有 $4 \times 5 \times 5 = 100$ 次求和计算以及 $4 \times 5 \times 4 = 80$ 次比较运算，最后阶段 0 从点 0 到 1、2、3、4、5 共有

5 次计算即可探索清晰总的路径信息，选取最短路径只需再进行 4 次比较，因此总的计算次数为 $100+5=105$ 次求和计算以及 $80+4=84$ 次比较。这些计算仅仅占据枚举法的 $189/18749 \approx 0.01$，因此动态规划在较大规模问题上具有显著的优势。

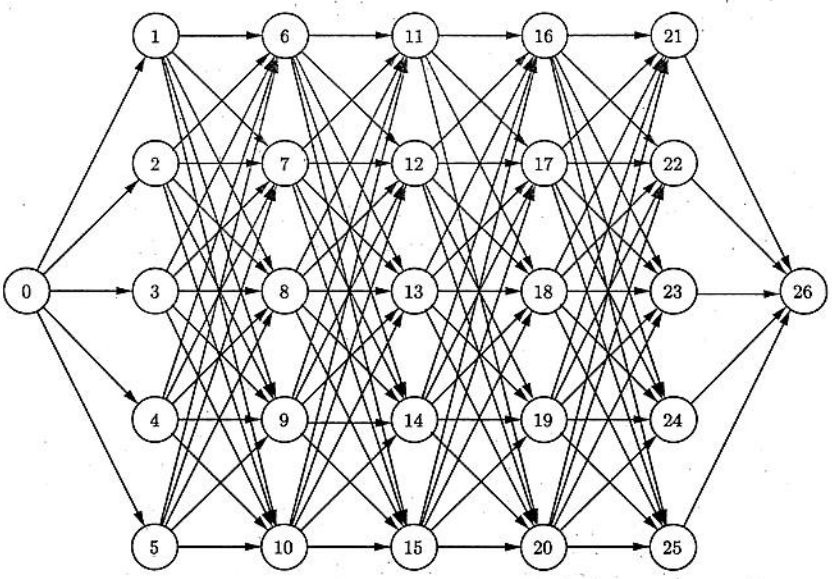


图 14.3 最短路：一个较大规模的网络

14.2 动态规划的实现

14.2.1 伪代码

下面给出用动态规划求解上述例子的伪代码和代码。首先给出伪代码如下。

Algorithm 12 Dynamic Programming 算法

Input: 网络中的孤矩阵

Output: 最短距离和路径信息

1: 输入网络数据，初始化 nodeNum, stageNum, nodeNumInOneStage
2: 定义数据结构 PathData，其包含 pathInfo，pathDistance 等
3: 初始化：pathList $\leftarrow \varnothing$
4: **for** int i =nodeNum−1−nodeNumInOneStage; i <nodeNum−1; $i++$ **do**
5: t=stageNum
6: new PathData pd
7: pd.pathDistance=$f_t(i) = c_{i,\text{nodeNum}-1}$ and pd.pathInfo.add(nodeNum−1)
8: pathList.add(pd)
9: **end for**

10: **for** int t =stageNumber$-1; t > 0; t--$ **do**
11: **for** int i =nodeNumInOneStage$*t - (nodeNumberInOneStage - 1)$;
12: i <nodeNumInOneStage$*t + 1; i++$ **do**
13: new PathData pd1
14: pd1.pathDistance=$f_t(i) = \min\{f_{t+1}(j) + c_{ij}\}$,
15: $\forall j \in \{$nodeNumInOneStage$*t + 1, \cdots,$nodeNumInOneStage$*t + 5\}$
16: pd1.pathInfo.add(i)
17: pathList.add(pd1)
18: **end for**
19: **end for**
20: **for** int $i = 1; i <$nodeNumInOneStage$+1; i++$ **do**
21: new PathData pd2
22: pd2.pathDistance=$f_0(0) = \min\{f_1(i) + c_{0i}\}$ and pd2.pathInfo.add(i)
23: pathList.add(pd2)
24: **end for**
25: **return** pathList.get(lastElement).pathDistance & pathInfo

14.2.2 Java 代码

1. PathData 类

先构建路径拓展所需的数据结构，即 PathData 类，代码如下：

Path Data.java

```
/**
* @author: Yongsen Zang
* @School: Tsinghua University
* @操作说明：动态规划求解最短路问题之定义数据结构
*
*/
package dynamicProgrammingForSP;
import java.util.ArrayList;

public class PathData {
        int t;
        int nodeID;
        double pathDistance;
        ArrayList<Integer> pathInfo = new ArrayList<>();
}
```

2. GenerateRandomData 类

由于网络图没有给定各弧段的长度，我们为每个弧段随机生成一个长度，即 GenerateRandomData 类，代码如下：

GenerateRandomData.java

```
/**
* @author: Yongsen Zang
* @School: Tsinghua University
* @操作说明：动态规划求解最短路问题之生成随机矩阵
*
*/
package dynamicProgrammingForSP;
import java.math.BigDecimal;

public class GenerateRandomData {

        public static double[][] GenerateDistanceMatrix(int nodeNumber, int stageNumber, int nodeNumberInOneStage){
                double arcDistance[][] = new double[nodeNumber][nodeNumber];

                for(int i=0; i<nodeNumber; i++){
                        for(int j=0; j<nodeNumber; j++){
                                arcDistance[i][j] = Integer.MAX_VALUE;
                        }
                }
                for(int t=1; t<stageNumber; t++){
                        for(int i=nodeNumberInOneStage*t-(nodeNumberInOneStage-1); i<=nodeNumberInOneStage*t; i++){
                                for(int j=nodeNumberInOneStage*t+1; j<=nodeNumberInOneStage*t+nodeNumberInOneStage; j++){
                                        arcDistance[i][j] = GenerateRandomArcDistance();
                                }
                        }
                }
                for(int i=1; i<nodeNumberInOneStage+1; i++){
                        arcDistance[0][i] = GenerateRandomArcDistance();
                        arcDistance[nodeNumberInOneStage*(stageNumber-1)+i][nodeNumber-1] = GenerateRandomArcDistance();
                }
                return arcDistance;
        }
```

```
33
34          /**
35          * 生成300～1500的随机数
36          * @return 随机数
37          */
38          public static double GenerateRandomArcDistance() {
39                  int max=1500,min=300;
40                  double ran = (Math.random()*(max-min)+min);
41                  BigDecimal b = new BigDecimal(ran);
42                  ran = b.setScale(2, BigDecimal.ROUND_HALF_UP).doubleValue();
43                  return ran;
44          }
45
46  }
```

3. DynamicProgrammingForSP 类

动态规划求解的主要过程，即 main 方法，代码如下：

DynamicProgrammingForSP.java

```
1  /**
2  * @author: Yongsen Zang
3  * @School: Tsinghua University
4  * @操作说明：动态规划求解最短路问题
5  *
6  */
7  package dynamicProgrammingForSP;
8  import java.util.ArrayList;
9
10 public class DynamicProgrammingForSP {
11
12          static int nodeNumber = 27;
13          static int stageNumber = 5;
14          static int nodeNumberInOneStage = 5;
15          static double[][] cij = new double[nodeNumber][nodeNumber];
16          public static void main(String[] args){
17                  //获取算例数据
18                  cij = GenerateRandomData.GenerateDistanceMatrix(nodeNumber,
        stageNumber, nodeNumberInOneStage);
19                  for(int i=0; i<nodeNumber; i++){
20                          for(int j=0; j<nodeNumber; j++){
21                                  System.out.print(cij[i][j]+ ", ");
22                          }
```

```
                System.out.println();
            }

            //定义一个ArrayList用来存储各个阶段生成的路径信息
            ArrayList<PathData> pathList = new ArrayList<PathData>();

            //按阶段进行后向动态规划
            //先进行最后一个阶段，即第5阶段
            for(int i=nodeNumber-1-nodeNumberInOneStage; i<nodeNumber-1; i++){
                PathData pada = new PathData();
                pada.t = stageNumber;
                pada.nodeID = i;
                pada.pathDistance = cij[i][nodeNumber-1];//nodeNumber-1==26 in this example
                pada.pathInfo.add(nodeNumber-1);
                pada.pathInfo.add(i);
                pathList.add(pada);
            }
            //进行第4阶段到第1阶段的递推计算
            for(int t=stageNumber-1; t>0; t--){
                for(int i=nodeNumberInOneStage*t-(nodeNumberInOneStage-1); i<nodeNumberInOneStage*t+1; i++){
                    PathData pd = new PathData();
                    pd = FindShortestSubpath(pathList, i, t);
                    pathList.add(pd);
                }
            }
            //进行第0阶段的计算，即从点0到点1～5的计算
            for(int i=1; i<nodeNumberInOneStage+1; i++){
                PathData pd = FindShortestSubpath(pathList, 0, 0);
                pathList.add(pd);
            }
            //输出最终结果
            System.out.println("The shortest path distance is: " + pathList.get(pathList.size()-1).pathDistance);
            System.out.print("The shortest path is: ");
            for(int i=pathList.get(pathList.size()-1).pathInfo.size()-1; i>=0; i--){
                System.out.print(pathList.get(pathList.size()-1).pathInfo.get(i) + "->");
            }
```

```
59          }
60
61          /**
62          * 计算每个出发节点到达目的地的最短路径，当前出发节点就是要拓展的节点
63          * @param pathList
64          * @param i，当前要扩展的节点
65          * @param t，当前的阶段
66          * @return PathData
67          */
68          public static PathData FindShortestSubpath(ArrayList<PathData>
        pathList, int i, int t){
69
70                  PathData pathdata = new PathData();
71                  double newLength = 0;
72                  int nodeID = 0;
73                  ArrayList<Integer> jpath = new ArrayList<Integer>();
74                  ArrayList<Integer> extendPath = new ArrayList<Integer>();
75                  double agency = Integer.MAX_VALUE;
76                  for(int j=nodeNumberInOneStage*t+1; j<nodeNumberInOneStage*t
        +6; j++){
77                          double subdis =0;
78
79                          for(PathData pa: pathList){
80                                  if(pa.nodeID == j){
81                                          subdis = pa.pathDistance;
82                                          jpath = pa.pathInfo;
83                                          break;
84                                  }
85                          }
86
87                          newLength = subdis + cij[i][j];
88                          if(newLength <= agency){
89                                  agency = newLength;
90                                  nodeID = i;
91                                  extendPath = DeepCopy(jpath);
92                          }
93                  }
94                  pathdata.nodeID = nodeID;
95                  pathdata.pathDistance = agency;
96                  for(int node: extendPath){
97                          pathdata.pathInfo.add(node);
98                  }
```

```
                pathdata.pathInfo.add(nodeID);
                pathdata.t = t;

                return pathdata;
        }

        //深拷贝路径信息的方法
        public static ArrayList<Integer> DeepCopy(ArrayList<Integer> arr){
                ArrayList<Integer> newArr = new ArrayList<Integer>();
                for(int a: arr){
                        newArr.add(a);
                }
                return newArr;
        }

}
```

注意：需要注意的是，在实际应用中，有些时候会遇到不太容易划分阶段的情况，这时也可以不进行阶段的划分，但在确定状态转移方程时会稍微复杂点，同时所需的计算次数也会相应增加，即计算复杂度可能会有一定的增加，如果读者感兴趣，可以自己尝试不划分阶段求解上述最短路问题，这里不做详细阐述。此外，本章只是抛砖引玉地针对最短路问题设计了动态规划算例进行讲解，其应用场景还有很多，比如背包问题、TSP、排程问题等，感兴趣的读者可以自行探索研究。

14.3 动态规划求解 TSP

上文我们介绍了动态规划求解最短路问题的基本原理和代码实现。本节我们以 TSP 问题为例，继续深入探讨动态规划。

14.3.1 一个简单的 TSP 算例

我们以一个非常简单的例子开始。考虑有 4 个点的网络（例子和原理介绍参考自 https://www.youtube.com/watch?v=XaXsJJh-Q5Y），如图 14.4 所示。

其对应的距离矩阵为

$$\boldsymbol{D}=\begin{bmatrix}0 & 8 & 12 & 17\\ 4 & 0 & 10 & 9\\ 7 & 14 & 0 & 13\\ 9 & 7 & 11 & 0\end{bmatrix}$$

注意：上述距离矩阵不是对称的。

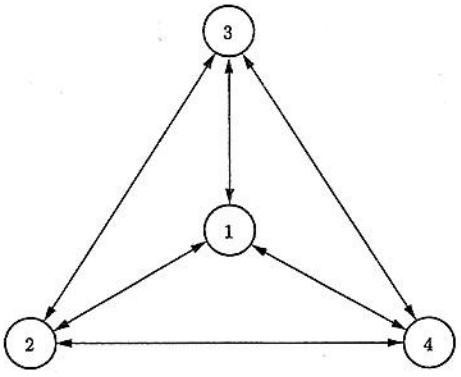


图 14.4 TSP：一个小例子

如果我们把点 1 作为起始点，枚举所有的路径，就如图 14.5 中展示的那样。如果从上到下枚举，复杂度为 $O(n!)$，其中 n 为顾客点的个数。

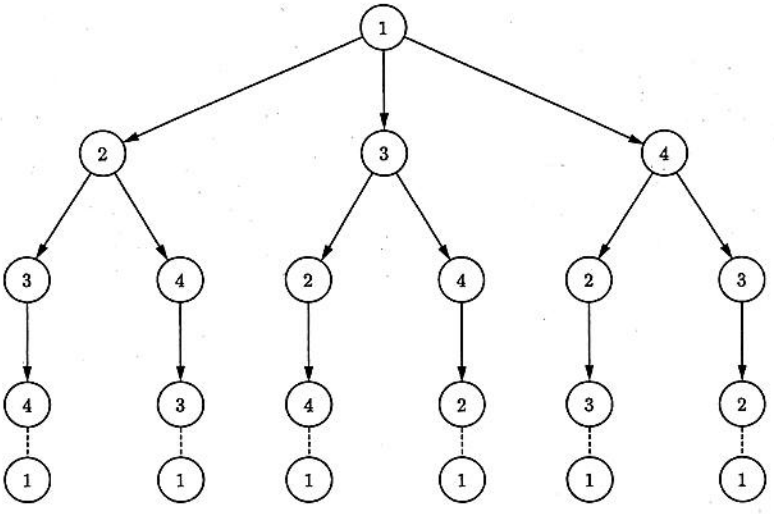


图 14.5 TSP 小例子：枚举所有路径

但是如果我们从下往上逆推，问题的复杂度就会有所不同。容易得出，当顾客点为 4 的时候，任意一个解对应的路径中的节点个数为 5，例如 $1 \to 2 \to 4 \to 3 \to 1$。另外，任意一个解，其最后到达的点一定是起点 1。因此我们可以将整个过程分为 5 个阶段，每个阶段的状态，就是该阶段所在的顾客点。

第 5 阶段，最终到达的一定是点 1。

第 4 阶段，剩余还没有访问的点集就是 $\{1\}$，但是在第 4 阶段当前所处的位置却有 3 种不同的情况，即 $\{2,3,4\}$。如果第 4 阶段在点 4，则从 4 出发，到达终点 1 的距离为 $d_{4,1}=9$；同理，从 2 和 3 出发的距离分别为 $d_{2,1}=4$ 和 $d_{3,1}=7$。图 14.6 中第 4 层虽然有 6 个节点，但实际上我们只计算 3 次。

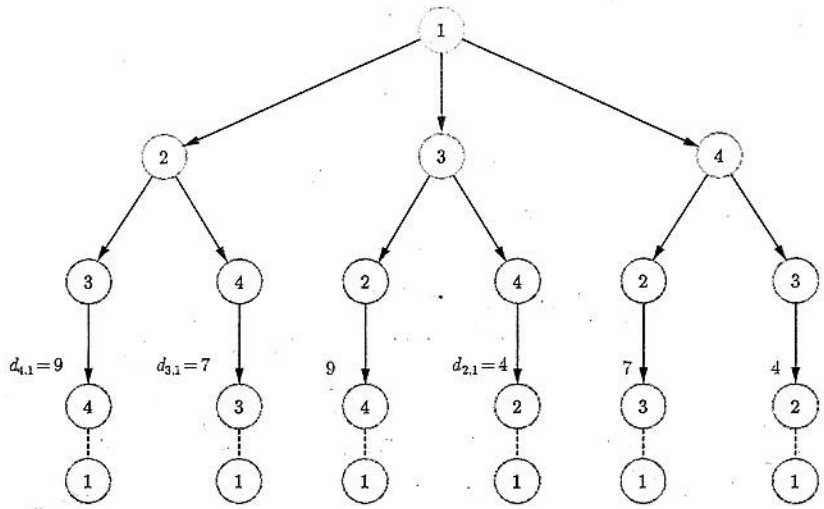


图 14.6 TSP 小例子：第 4 阶段

第 3 阶段，如图 14.7 所示。剩余还没有访问的点集就有所不同。当前所在的点 $v_c \in \{2,3,4\}$。并且剩余还没访问的点的个数为 2，其中有一个点是 1，因此我们考虑除了 1 以外，还没有被访问的点的集合。如果 $v_c=3$，除了 1 以外，还没有被访问的点的集合大小为 1，集合 S_l 可以为 $\{2\}$ 或者 $\{4\}$。

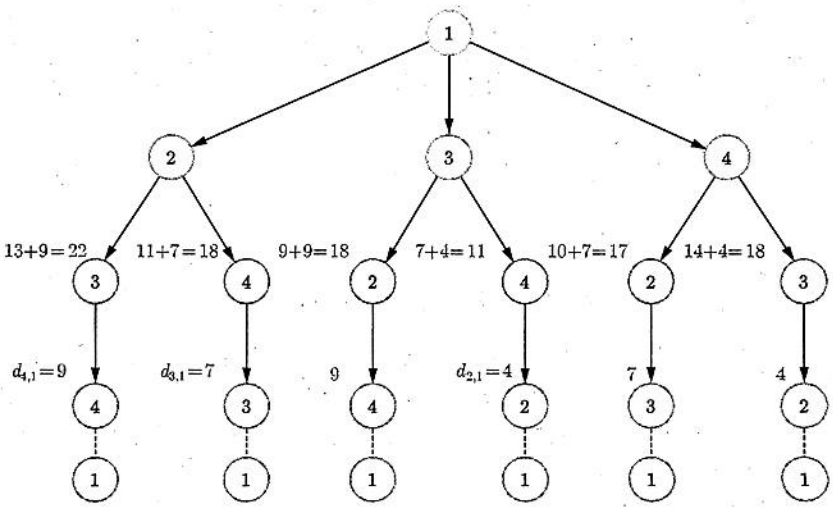


图 14.7 TSP 小例子：第 3 阶段

（1）如果 $v_c=3, S_l=\{4\}$，则第 3 个点访问顾客点 3，然后经过顾客点 4，回到顾客点 1 的最小距离为 $d_{3,4}+d_{4,1}=13+9=22$。

（2）如果 $v_c=4, S_l=\{3\}$，则第 3 个点访问顾客点 4，然后经过顾客点 3，回到顾客

点 1 的最小距离为 $d_{4,3}+d_{3,1}=11+7=18$。

（3）如果 $v_c=2, S_l=\{4\}$，则第 3 个点访问顾客点 2，然后经过顾客点 4，回到顾客点 1 的最小距离为 $d_{2,4}+d_{4,1}=9+9=18$。

（4）如果 $v_c=4, S_l=\{2\}$，则第 3 个点访问顾客点 4，然后经过顾客点 2，回到顾客点 1 的最小距离为 $d_{4,2}+d_{2,1}=7+4=11$。

（5）如果 $v_c=2, S_l=\{3\}$，则第 3 个点访问顾客点 2，然后经过顾客点 3，回到顾客点 1 的最小距离为 $d_{2,3}+d_{3,1}=10+7=17$。

（6）如果 $v_c=3, S_l=\{2\}$，则第 3 个点访问顾客点 3，然后经过顾客点 2，回到顾客点 1 的最小距离为 $d_{3,2}+d_{2,1}=14+4=18$。

第 2 阶段，如图 14.8 所示。当前所在的点 $v_c\in\{2,3,4\}$。并且剩余还没访问的点（除了顾客点 1 以外）的个数为 2。

（1）如果 $v_c=2, S_l=\{3,4\}$，即第 2 个点访问顾客点 2，然后以任意顺序访问顾客集合 {3 4}，回到顾客点 1 的最小距离为 $d^2_{\min}=\min\{10+22,9+18\}=27$。

（2）如果 $v_c=3, S_l=\{2,4\}$，即第 2 个点访问顾客点 2，然后以任意顺序访问顾客集合 {2 4}，回到顾客点 1 的最小距离为 $d^3_{\min}=\min\{14+18,13+11\}=24$。

（3）如果 $v_c=4, S_l=\{2,3\}$，即第 2 个点访问顾客点 2，然后以任意顺序访问顾客集合 {2,3}，回到顾客点 1 的最小距离为 $d^4_{\min}=\min\{7+17,11+18\}=24$。

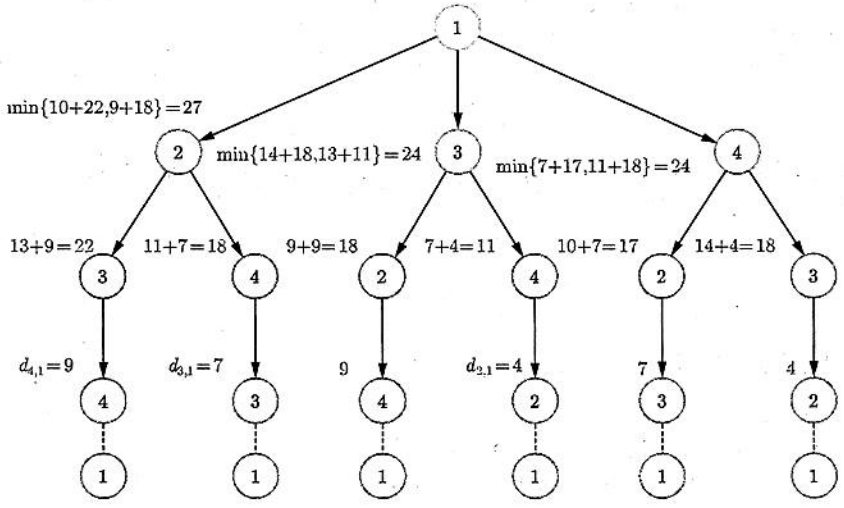


图 14.8 TSP 小例子：第 2 阶段

第 1 阶段，如图 14.9 所示。当前所在的点只能是点 1，因此 $v_c=1$。并且剩余还没访问的点（除了顾客点 1 以外）的个数为 3，集合为 $S_l=\{2,3,4\}$，因此最小距离为 $d_{\min}=\min\{d^2_{\min}+d_{1,2}, d^3_{\min}+d_{1,3}, d^4_{\min}+d_{1,4}\}=\min\{8+27,12+24,17+24\}=35$。

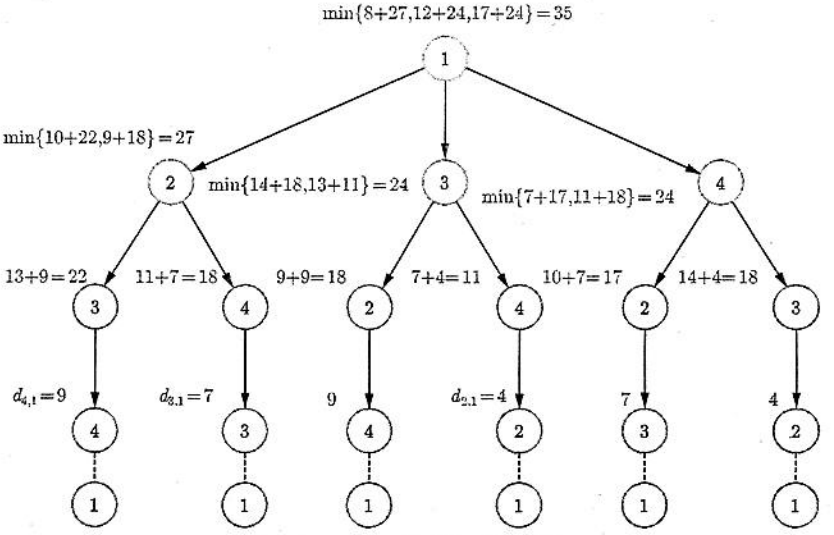


图 14.9 TSP 小例子：第 1 阶段

接下来我们就可以给出通项公式。我们用 $f(v_c, S_l)$ 表示当前节点为 v_c、剩余未访问的节点集合为 S_l 时的最小距离，则有

$$f(1,\{2,3,4\}) = \min\{d_{1,k} + f(k,\{2,3,4\} - \{k\}), \quad k = 2,3,4\}$$

更一般地，有

$$f(i,S) = \min_{k \in S}\{d_{i,k} + f(k, S - \{k\})\} \tag{14.1}$$

接下来我们用上述递推公式来求解一遍上述例子。我们从第 4 阶段开始，$v_c \in \{2,3,4\}$，$S_l = \varnothing$，则计算如下：

（1）$f(2,\varnothing) = d_{2,1} = 4$；

（2）$f(3,\varnothing) = d_{3,1} = 7$；

（3）$f(4,\varnothing) = d_{4,1} = 9$。

第 3 阶段，$v_c \in \{2,3,4\}$，则分下面的情况讨论。

（1）$v_c = 2$，$S_l = \{3\}$ 或 $\{4\}$：

① $f(2,\{3\}) = d_{2,3} + \min\{f(3,\varnothing)\} = 10 + 7 = 17$；

② $f(2,\{4\}) = d_{2,4} + \min\{f(4,\varnothing)\} = 9 + 9 = 18$。

（2）$v_c = 3$，$S_l = \{2\}$ 或 $\{4\}$：

① $f(3,\{2\}) = d_{3,2} + \min\{f(2,\varnothing)\} = 14 + 4 = 18$；

② $f(3,\{4\}) = d_{3,4} + \min\{f(4,\varnothing)\} = 13 + 9 = 22$。

（3）$v_c = 4$，$S_l = \{2\}$ 或 $\{3\}$：

① $f(4,\{2\}) = d_{4,2} + \min\{f(2,\varnothing)\} = 7 + 4 = 11$；

② $f(4,\{3\})=d_{4,3}+\min\{f(3,\varnothing)\}=11+7=18$。

第 2 阶段，$v_c\in\{2,3,4\}$，则分下面的情况讨论：

（1）$v_c=2$，$S_l=\{3,4\}$，则 $f(2,\{3,4\})=\min\{d_{2,3}+f(3,\{4\}),d_{2,4}+f(4,\{3\})\}=\min\{10+22,9+18\}=27$；

（2）$v_c=3$，$S_l=\{2,4\}$，则 $f(3,\{2,4\})=\min\{d_{3,2}+f(2,\{4\}),d_{3,4}+f(4,\{2\})\}=\min\{14+18,13+11\}=24$；

（3）$v_c=4$，$S_l=\{2,3\}$，则 $f(4,\{2,3\})=\min\{d_{4,2}+f(2,\{3\}),d_{4,3}+f(3,\{2\})\}=\min\{7+17,11+18\}=24$。

第 1 阶段，$v_c=1$，则计算如下：

$$\begin{aligned}f(1,\{2,3,4\})&=\min\{d_{1,2}+f(2,\{3,4\}),\quad d_{1,3}+f(3,\{2,4\}),\quad d_{1,4}+f(4,\{2,3\})\}\\&=\min\{8+27,\quad 12+24,\quad 17+24\}=35。\end{aligned}$$

至于最优路径，我们顺着第 1 阶段向第 5 阶段回溯，可以得到，最优路径为 $1\to2\to4\to3\to1$。

可以明显看出，在上述过程中，我们在第 4 到第 1 阶段的计算次数共为 3+6+6+1 = 16 次。而如果用排列组合的方法，计算次数为 $4!=24$ 次。显然，使用动态规划的方法，显著降低了计算复杂度。经过分析，不难得出，动态规划求解 TSP 的复杂度为 $O(n^2\cdot2^n)$。非常可惜，虽然动态规划降低了求解复杂度，但是从本质上来讲，动态规划求解 TSP 仍旧是一个指数时间复杂度的算法。

14.3.2 伪代码

前面章节介绍了动态规划求解 TSP 的原理。本节和下一节，我们来探讨如何将其实现。首先，我们根据之前的理论介绍，整理出动态规划求解 TSP 的伪代码。

Algorithm 13 动态规划求解 TSP，算法复杂度 $O(n^2\cdot2^n)$

Input: 图 $G=(V,A)$，起始节点 $s=1$

Output: 最优路径 r^*，其中每个节点均被访问且只被访问一次

```
for i = 2,3,⋯,|V| do
    f(i,∅) ← d_{i,s}
    (i,∅) 的下一个节点 ← s
end for
for k = V, V−1, V−2, ⋯, 2 do
    for 当前节点 i = 2,3,⋯,|V| do
        for S_l ⊂ V\{i} 和 |S_l| = |V| − k do
            f(i,S_l) = min_{v∈S_l}{d_{i,v} + f(v, S_l − {v})}
            (i,S_l) 的下一个节点 ← arg min_v min_{v∈S_l}{d_{i,v} + f(v, S_l − {v})}
        end for
```

11: end for
12: **end for**
13: $S_l \leftarrow V - \{s\}$
14: $f(s, S_l) \leftarrow \min_{v \in S_l}\{d_{s,v} + f(v, S_l - \{v\})\}$
15: (s, S_l) 的下一个节点 $\leftarrow \arg\min_{v} \min_{v \in S_l}\{d_{s,v} + f(v, S_l - \{v\})\}$
16: 通过检查当前节点的下一个节点，获得最优路径 r^*
17: **return** 最优路径 r^* 和最优目标值 $f(s, S_l)$

14.3.3 Python 实现：示例算例

Python 实现动态规划求解 TSP 的代码如下：

TSP Dynamic Programming.py

```
'''
    Author       : Liu Xinglu
    Institute    : Tsinghua University
    Date         : 2020-10-21
'''

import copy
import re
import math
import itertools

Nodes = [1, 2, 3, 4]

nodeNum = len(Nodes)

dis_matrix = [[0, 8, 12, 17]
              , [4, 0, 10, 9]
              , [7, 14, 0, 13]
              , [9, 7, 11, 0]  ]


def TSP_Dynamic_Programming(Nodes, dis_matrix):
    Label_set = {}
    nodeNum = len(Nodes)
    org = 1
    # cycle stage : V, V-1, ..., 2
    for stage_ID in range(nodeNum, 1, -1):  # 逆序遍历stage_ID
        print('stage :', stage_ID)
```

```
        for i in range(2, nodeNum + 1):
            current_node = i
            left_node_list = copy.deepcopy(Nodes)
            left_node_list.remove(i)
            if (org in left_node_list):
                left_node_list.remove(org)
            left_node_set = set(left_node_list)
            # obtain the all the subset of the left node set
            subset_all = list(map(set, itertools.combinations(left_node_set,
        nodeNum - stage_ID)))
            # print('current_node:', current_node, '\t left_node_set : ',
        # left_node_set)
            for subset in subset_all:
                if (len(subset) == 0):
                    key = (stage_ID, current_node, 'None')
                    next_node = org
                    Label_set[key] = [dis_matrix[i - 1][org - 1], next_node]
                else:
                    key = (stage_ID, current_node, str(subset))
                    min_distance = 1000000
                    for temp_next_node in subset:
                        subsub_set = copy.deepcopy(subset)
                        subsub_set.remove(temp_next_node)
                        if (subsub_set == None or len(subsub_set) == 0):
                            subsub_set = 'None'
                        sub_key = (stage_ID + 1, temp_next_node, str(subsub_
        set))
                        if (sub_key in Label_set.keys()):
                            if (dis_matrix[current_node - 1][temp_next_node
        - 1] + Label_set[sub_key][0] < min_distance):
                                min_distance = dis_matrix[current_node - 1][
        temp_next_node - 1] + Label_set[sub_key][0]
                                next_node = temp_next_node
                                Label_set[key] = [min_distance, next_node]

            # print('current_node:', current_node, '\t left_node_set : ',
         # subset)

            # stage 1 :
    stage_ID = 1
    current_node = org
    subset = set(copy.deepcopy(Nodes))
```

```
    subset.remove(org)
    final_key = (stage_ID, current_node, str(subset))
    min_distance = 1000000
    for temp_next_node in subset:
        subsub_set = copy.deepcopy(subset)
        subsub_set.remove(temp_next_node)
        if (subsub_set == None or len(subsub_set) == 0):
            subsub_set = 'None'
        sub_key = (stage_ID + 1, temp_next_node, str(subsub_set))
        if (sub_key in Label_set.keys()):
            if (dis_matrix[current_node - 1][temp_next_node - 1] + Label_set
       [sub_key][0] < min_distance):
                min_distance = dis_matrix[current_node - 1][temp_next_node -
        1] + Label_set[sub_key][0]
                next_node = temp_next_node
                Label_set[final_key] = [min_distance, next_node]

    # get the optimal solution
    opt_route = [org]
    not_visted_node = set(copy.deepcopy(Nodes))
    not_visted_node.remove(org)
    next_node = Label_set[final_key][1]
    while(True):
        opt_route.append(next_node)
        if(len(opt_route) == nodeNum + 1):
            break
        current_stage = len(opt_route)
        not_visted_node.remove(next_node)
        if (not_visted_node == None or len(not_visted_node) == 0):
            not_visted_node = 'None'
        next_key = (current_stage, next_node, str(not_visted_node))
        next_node = Label_set[next_key][1]

    opt_dis = Label_set[final_key][0]

    print('objective :', Label_set[final_key][0])
    print('optimal route :', opt_route)

    return opt_dis, opt_route


# call function to solve TSP
```

```
opt_dis, opt_route = TSP_Dynamic_Programming(Nodes, dis_matrix)
print('\n\n ---------  optimal solution  ---------\n')
print('objective :', opt_dis)
print('optimal route :', opt_route)
```

运行结果如下：

result

```
---------  optimal solution  ---------

objective : 35
optimal route : [1, 2, 4, 3, 1]
```

14.3.4 Python 实现：中大规模算例

我们以 Solomon 的 VRP 标杆算例来测试中大规模的情况。完整代码如下：

Dynamic Programming for Solving TSP

```
#  * author :  Liu Xinglu
#  * Institute: Tsinghua University
#  * Date : 2020-9-15
#  * E-mail : hsinglul@163.com

# # Read data Function
import pandas as pd
import numpy as np
import networkx as nx
import matplotlib.pyplot as plt
import copy
import re
import math
import itertools

class Data:
    customerNum = 0;
    nodeNum     = 0;
    vehicleNum  = 0;
    capacity    = 0;
    cor_X       = [];
    cor_Y       = [];
    demand      = [];
    serviceTime = [];
```

```
    readyTime    = [];
    dueTime      = [];
    disMatrix    = [[]]; # 读取数据

# function to read data from .txt files
def readData(data, path, customerNum):
    data.customerNum = customerNum;
    data.nodeNum = customerNum + 1;
    # data.nodeNum = customerNum + 2;
    f = open(path, 'r');
    lines = f.readlines();
    count = 0;
    # read the info
    for line in lines:
        count = count + 1;
        if(count == 5):
            line = line[:-1].strip();
            str = re.split(r" +", line);
            data.vehicleNum = int(str[0]);
            data.capacity = float(str[1]);
        elif(count >= 10 and count <= 10 + customerNum):
            line = line[:-1];
            str = re.split(r" +", line);
            data.cor_X.append(float(str[2]));
            data.cor_Y.append(float(str[3]));
            data.demand.append(float(str[4]));
            data.readyTime.append(float(str[5]));
            data.dueTime.append(float(str[6]));
            data.serviceTime.append(float(str[7]));

    # compute the distance matrix
    data.disMatrix = [([0] * data.nodeNum) for p in range(data.nodeNum)];
       # 初始化距离矩阵的维度,防止浅拷贝
    # data.disMatrix = [[0] * nodeNum] * nodeNum]; 这个是浅拷贝，容易重复
    for i in range(0, data.nodeNum):
        for j in range(0, data.nodeNum):
            temp = (data.cor_X[i] - data.cor_X[j])**2 + (data.cor_Y[i] -
      data.cor_Y[j])**2;
            data.disMatrix[i][j] = round(math.sqrt(temp), 1);
            # if(i == j):
            # data.disMatrix[i][j] = 0;
            # print("%6.2f" % (math.sqrt(temp)), end = " ");
```

```
            temp = 0;

    return data

def printData(data, customerNum):
    print("下面打印数据\n");
    print("vehicle number = %4d" % data.vehicleNum);
    print("vehicle capacity = %4d" % data.capacity);
    for i in range(len(data.demand)):
        print('{0}\t{1}\t{2}\t{3}'.format(data.demand[i], data.readyTime[i],
      data.dueTime[i],  data.serviceTime[i]));

    print("-------距离矩阵-------\n");
    for i in range(data.nodeNum):
        for j in range(data.nodeNum):
            #print("%d   %d" % (i, j));
            print("%6.2f" % (data.disMatrix[i][j]), end = " ");
        print()

# # Read data
data = Data()
path = 'Solomn标准VRP算例/solomon-100/In/r101.txt'
customerNum = 20
readData(data, path, customerNum)
printData(data, customerNum)

# # Build Graph
# 构建有向图对象
Graph = nx.DiGraph()
cnt = 0
pos_location = {}
nodes_col = {}
nodeList = []
for i in range(data.nodeNum):
    X_coor = data.cor_X[i]
    Y_coor = data.cor_Y[i]
    name = str(i)
    nodeList.append(name)
    nodes_col[name] = 'gray'
    node_type = 'customer'
    if(i == 0):
        node_type = 'depot'
```

```
    Graph.add_node(name
                , ID = i
                , node_type = node_type
                , time_window = (data.readyTime[i], data.dueTime[i])
                , arrive_time = 10000   # 这个是时间标签1
                , demand = data.demand
                , serviceTime = data.serviceTime
                , x_coor = X_coor
                , y_coor = Y_coor
                , min_dis = 0           # 这个是距离标签2
                , previous_node = None  # 这个是前序节点标签3
                )

    pos_location[name] = (X_coor, Y_coor)
# add edges into the graph
for i in range(data.nodeNum):
    for j in range(data.nodeNum):
        if(i != j):
            Graph.add_edge(str(i), str(j)
                                    , travelTime = data.disMatrix[i][j]
                                    , length = data.disMatrix[i][j]
                                   )

# plt.rcParams['figure.figsize'] = (0.6 * trip_num, trip_num) # 单位是inch
nodes_col['0'] = 'red'
# nodes_col[str(data.nodeNum-1)] = 'red'
plt.rcParams['figure.figsize'] = (10, 10) # 单位是inch
nx.draw(Graph
        , pos=pos_location
        # , with_labels = True
        , node_size = 50
        , node_color = nodes_col.values()   #'y'
        , font_size = 15
        , font_family = 'arial'
        # , edge_color = 'grey'   #'grey'  # b, k, m, g,
        , edgelist = []
        # , nodelist = nodeList
       )
fig_name = 'network_' + str(customerNum) + '_1000.jpg'
plt.savefig(fig_name, dpi=600)
plt.show()
```

```
# # Dynamic Programming
Nodes = list(Graph.nodes())
for i in range(len(Nodes)):
    Nodes[i] = (int)(Nodes[i]) + 1

dis_matrix = np.zeros([len(Nodes), len(Nodes)])
for i in range(len(Nodes)):
    for j in range(len(Nodes)):
        if(i != j):
            key = (str(i), str(j))
            dis_matrix[i][j] = Graph.edges[key]['length']

def TSP_Dynamic_Programming(Nodes, dis_matrix):
    Label_set = {}
    nodeNum = len(Nodes)
    org = 1
    # cycle stage : V, V-1, ..., 2
    for stage_ID in range(nodeNum, 1, -1):  # 逆序遍历stage_ID
        print('stage :', stage_ID)
        for i in range(2, nodeNum + 1):
            current_node = i
            left_node_list = copy.deepcopy(Nodes)
            left_node_list.remove(i)
            if (org in left_node_list):
                left_node_list.remove(org)
            left_node_set = set(left_node_list)
            # obtain the all the subset of the left node set
            subset_all = list(map(set, itertools.combinations(left_node_set,
        nodeNum - stage_ID)))
            # print('current_node:', current_node, '\t left_node_set : ',
       # left_node_set)
            for subset in subset_all:
                if (len(subset) == 0):
                    key = (stage_ID, current_node, 'None')
                    next_node = org
                    Label_set[key] = [dis_matrix[i - 1][org - 1], next_node]
                else:
                    key = (stage_ID, current_node, str(subset))
                    min_distance = 1000000
                    for temp_next_node in subset:
                        subsub_set = copy.deepcopy(subset)
                        subsub_set.remove(temp_next_node)
```

```
                        if (subsub_set == None or len(subsub_set) == 0):
                            subsub_set = 'None'
                        sub_key = (stage_ID + 1, temp_next_node, str(subsub_
    set))
                        if (sub_key in Label_set.keys()):
                            if (dis_matrix[current_node - 1][temp_next_node
    - 1] + Label_set[sub_key][0] < min_distance):
                                min_distance = dis_matrix[current_node - 1][
    temp_next_node - 1] + Label_set[sub_key][0]
                                next_node = temp_next_node
                                Label_set[key] = [min_distance, next_node]

            # print('current_node:', current_node, '\t left_node_set : ',
    # subset)

    # stage 1 :
    stage_ID = 1
    current_node = org
    subset = set(copy.deepcopy(Nodes))
    subset.remove(org)
    final_key = (stage_ID, current_node, str(subset))
    min_distance = 1000000
    for temp_next_node in subset:
        subsub_set = copy.deepcopy(subset)
        subsub_set.remove(temp_next_node)
        if (subsub_set == None or len(subsub_set) == 0):
            subsub_set = 'None'
        sub_key = (stage_ID + 1, temp_next_node, str(subsub_set))
        if (sub_key in Label_set.keys()):
            if (dis_matrix[current_node - 1][temp_next_node - 1] + Label_set
    [sub_key][0] < min_distance):
                min_distance = dis_matrix[current_node - 1][temp_next_node -
     1] + Label_set[sub_key][0]
                next_node = temp_next_node
                Label_set[final_key] = [min_distance, next_node]

    # get the optimal solution
    opt_route = [org]
    not_visted_node = set(copy.deepcopy(Nodes))
    not_visted_node.remove(org)
    next_node = Label_set[final_key][1]
    while(True):
```

```
        opt_route.append(next_node)
        if(len(opt_route) == nodeNum + 1):
            break
        current_stage = len(opt_route)
        not_visted_node.remove(next_node)
        if (not_visted_node == None or len(not_visted_node) == 0):
            not_visted_node = 'None'
        next_key = (current_stage, next_node, str(not_visted_node))
        next_node = Label_set[next_key][1]

    opt_dis = Label_set[final_key][0]

    print('objective :', Label_set[final_key][0])
    print('optimal route :', opt_route)

    return opt_dis, opt_route

opt_dis, opt_route = TSP_Dynamic_Programming(Nodes, dis_matrix)
print('\n\n ---------  optimal solution  ---------\n')
print('objective :', opt_dis)
print('optimal route :', opt_route)
```

上述代码中，我们以 R101 作为测试算例，取前 20 个点构建网络，求解了一个 21 个点的 TSP（加上 depot 是 21 个点）。运行结果如下：

result

```
stage : 21
stage : 20
stage : 19
stage : 18
stage : 17
stage : 16
stage : 15
stage : 14
stage : 13
stage : 12
stage : 11
stage : 10
stage : 9
stage : 8
stage : 7
stage : 6
stage : 5
```

```
stage : 4
stage : 3
stage : 2
objective : 262.9
optimal route : [1, 14, 7, 6, 18, 17, 15, 16, 3, 5, 13, 4, 10, 21, 2, 11,
       12,
20, 8, 9, 19, 1]
--------- optimal solution ---------
objective : 262.9
optimal route : [1, 14, 7, 6, 18, 17, 15, 16, 3, 5, 13, 4, 10, 21, 2, 11,
       12,
20, 8, 9, 19, 1]
```

14.4 标签算法求解带资源约束的最短路问题

前面章节介绍了动态规划求解 TSP 的原理，我们讲到，该方法仍旧是一个指数时间复杂度的算法，求解效率并不令人满意。对于 TSP，目前为止也没有能够求解超大规模算例的高效精确算法。但是对于一些其他路径规划相关的问题，却存在伪多项式时间的动态规划算法，或者较为高效的动态规划算法。本节需要讨论的带资源约束的最短路问题（Shortest Path Problem with Resource Constraints，SPPRC）就是一类存在伪多项式时间精确算法的问题。1986 年，Desrochers 博士在其博士论文中第一次提出 SPPRC（Desrochers，1986），之后，该问题被广泛地研究和拓展，也衍生出许多变种。特别地，SPPRC 的一个著名的变种，即带资源约束的基本最短路问题（Elementary Shortest Path Problem with Resource Constraints，ESPPRC），是 VRPTW 的列生成算法框架的子问题。ESPPRC 比 SPPRC 更难求解，因为 ESPPRC 要求路径中的节点至多只被访问一次，而 SPPRC 中允许同一个节点被访问多次。ESPPRC 已经被认定为强 NP-hard 问题（Dror，1994）。求解 SPPRC 和 ESPPRC 的动态规划算法有一个新的名称，即标签算法。在列生成算法的相应章节中，我们讲到，标签算法包括标签校正算法和标签设定算法（例如 Dijkstra）。标签算法本质上也是动态规划算法。本节介绍的标签算法属于标签校正算法。

14.4.1 带资源约束的最短路问题

本节以一个简单案例来介绍 SPPRC。考虑如图 14.10 所示的网络（例子来源于文献（Desaulniers et al.，2006））。其中，点 s 表示起点（source 或 origin），点 t 表示终点（sink 或 destination）。节点 i 上方的数组表示该点的最早开始服务时间 a_i 和最晚结束时间 b_i，即该点的时间窗为 $[a_i, b_i]$，例如点 1 的时间窗为 $[6, 14]$。图中弧 (i, j) 上的元组 (t_{ij}, c_{ij})，分别代表弧 (i, j) 上的行驶时间和成本（如距离等），例如，弧 $(s, 1)$ 的行驶时间和成本构成的元组为 $(8, 3)$。SPPRC 的目标就是找到一条从起点 s 出发到达终点 t 的成本最小的路径，并且该路径中途经的所有点都必须在其时间窗内被访问。

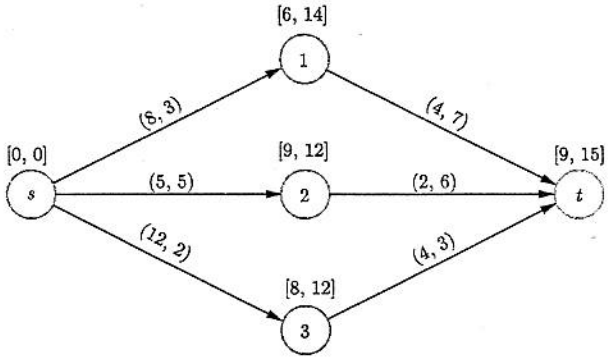


图 14.10 SPPRC 小例子

在上述 SPPRC 中，时间和成本可以看作两个资源，而考虑上述两个资源的 SPPRC，一般又叫作 Shortest Path Problem with Time Windows（SPPTW），该问题由 Desrosiers 等在 1983 年首次提出（Desrosiers et al.，1983；Desrosiers et al.，1984）。其中，时间资源是有限的，而成本资源是无限的。Desrosiers 等学者之后又将其进行了拓展，将其一般化为考虑多种资源（Desrochers，1986）的情形，在那之后，关于 SPPRC 的研究日益增多。例如，文献（Ioachim et al.，1998）提出了考虑节点的时间依赖成本的 SPPTW；文献（Dumas et al.，1991）研究了考虑搭载和配送因素的 SPPTW。

SPPRC 可以描述为：给定一个有向图 $G=(V,A)$，其中 $V=\{1,2,\cdots,N\}$ 是图中点的集合，$A=\{(i,j)|\forall i,j\in V,i\neq j\}$ 是图中弧的集合。$R=\{1,2,\cdots,K\}$ 表示所有资源的集合。G 中每条弧 (i,j) 都对应一个资源向量 $(r_1,r_2,\cdots,r_K)$，表示该条弧上的所有资源的消耗量。每个点 i 对应一个资源约束（例如时间窗）。给定起点 s 和终点 t，SPPRC 的目标就是找到一条从起点 s 出发到达终点 t 的成本最小的路径，并且该路径中途经的所有点都必须满足该点的资源约束。

这里以之前介绍过的 VRPTW 的子问题为例，给出 SPPRC 的整数规划模型。根据列生成相关章节的介绍，考虑时间、行驶成本和载重量 3 种资源的 SPPRC 的整数规划模型可以写成

$$\min \quad \sum_{i\in N}\sum_{j\in N}c_{ij}x_{ij} \tag{14.2}$$

$$\text{s.t.} \quad \sum_{i\in C}d_i\sum_{j\in N}x_{ij}\leqslant q \tag{14.3}$$

$$\sum_{j\in N}x_{0j}=1 \tag{14.4}$$

$$\sum_{i\in N}x_{ih}-\sum_{j\in N}x_{hj}=0, \qquad \forall h\in C \tag{14.5}$$

$$\sum_{i\in N}x_{i,n+1}=1 \tag{14.6}$$

$$s_i + t_{ij} - M(1 - x_{ij}) \leqslant s_j, \qquad \forall i, j \in N \tag{14.7}$$

$$a_i \leqslant s_i \leqslant b_i, \qquad \forall i \in N \tag{14.8}$$

$$x_{ij} \in \{0,1\}, \qquad \forall i, j \in N \tag{14.9}$$

根据上述模型，可以调用求解器求解 SPPRC。但是本章并不打算用求解器求解，而是用标签算法求解。

首先回到图 14.10。SPPRC 实际上就是在所有从起始点 s 出发到终点 t 的所有路径当中，选择一条最短的路径，并且满足一系列的资源约束。上述例子中，唯一的资源约束就是时间约束。在其他问题中，可能会有另外的资源，如载重、换班之间的停顿时间、搭载和配送约束等。这些资源都是随着路径的扩展，按照某种函数关系变化的，这种函数我们称为资源扩展函数（Resource Extension Function，REF）。在例子（图 14.10）中，我们用 T_i 表示在点 i 处时间资源的使用量（即到达点 i 的时间），则弧 (i,j) 的 REF 可以定义为

$$f_{ij}(T_i) = T_i + t_{ij}$$

其中，f_{ij} 就是弧 (i,j) 上的 REF。这个 REF 就是根据弧 (i,j) 中点 i 的访问时间，来更新点 j 的访问时间的函数。这个函数可以计算出在 T_i 时刻从点 i 出发，到达点 j 的最早时间。如果一条路径中，到达点 i 的时间超过了 b_i，则这个路径就不可行。但是，如果到达时间在 a_i 之前，则允许在点 i 等待一段时间。也就是说，到达点 i 的时间可以小于访问点 i 的时间。

在图 14.10 中，从起点到终点有 3 条路径。第 1 条路径 $P_1 = (s,1,t)$ 是资源可行的路径。的确，令 $T_s = 0$（在点 s 处的唯一可行值），我们很容易就可以得到在该条路径下，到达点 1 和 t 的时间（$T_1 = 8, T_t = 12$）。这两个值是根据所有的资源扩展函数（REF）得到的（$f_{s1}(T_s)$ 和 $f_{1t}(T_1)$）。

第 2 条路径 $P_2 = (s,2,t)$ 也是满足资源约束的路径。然而，在点 2 会有一些等待时间，到达点 2 的时间可以通过 $f_{s2}(T_s) = 5$ 计算，但是 $f_{s2}(T_s) = 5 < a_2 = 9$，在这种情况下，点 2 的访问时间可以设置成 $T_2 = 9$，并且在这条路径中，点 t 的访问时间可以设置成 $T_t = T_2 + 2 = 11$。

第 3 条路径 $P_2 = (s,3,t)$ 是资源不可行的，在这条路径上，$T_s = 0, T_3 = 12 \geqslant f_{s3} = 12$，到达点 t 的最早时间为 $f_{3t}(T_3) = f_{3t}(12) = 12 + 4 = 16$，由于点 t 的时间窗为 $[9,15]$，因此这条路径是不可行的。

考核 2 条可行路径 P_1, P_2，P_1 的成本为 $3 + 7 = 10$，比 P_2 的成本 $5 + 6 = 11$ 小，因此只考虑成本的话，P_1 是最优的。由于路径 P_2 有更早的到达时间（到达终点 t），如果图 14.10 只是一个大网络中的一个子网络，在拓展的时候，路径 P_2 有可能是一个可行路径，但是路径 P_1 有可能不是（由于 P_1 使用了更多的资源，可能导致后续拓展违背资源约束）。

以上描述让我们初步了解了 SPPRC 的难度。SPPRC 很接近于多标准问题。仅考虑单一标准的话，无法判断哪条路径是更好的，也就是很多时候，路径之间是不可比的，这也是 SPPRC 的困难之处。

不同类型的 SPPRC 可以根据下面的特征进行分类：

（1）资源累积的方式（不同的累积方式导致资源可行路径（Resource Feasible Path）定义不同）；

（2）是否存在额外的路径结构约束（Path-structural Constraint），如非基本路径、基本路径等；

（3）目标函数；

（4）基础网络结构。

这里用 $P=(v_0,v_1,\cdots,v_p)$ 表示一条路径，其中某个点可能会出现多次。路径 P 的长度为 p。接下来我们针对上面提出的 4 点来进行阐述。

1. 资源可行路径

在接下来的描述中，我们将区分资源的可行性与路径结构约束的可行性。资源约束可以通过资源消耗和资源间隔（也叫资源上下限、资源窗口等，例如 SPPTW 中的行驶时间 t_{ij} 和时间窗 $[a_i,b_i]$）来表示。设资源的数量为 R。向量 $\boldsymbol{T}=(T^1,T^2,\cdots,T^R)^{\mathrm{T}}\in\mathbb{R}^R$ 称为资源向量，对应的资源分配量就是决策变量，称为资源决策变量 $\boldsymbol{x}^{\mathrm{T}}$。所有资源的可用量为 $\boldsymbol{S}=(S^1,S^2,\cdots,S^R)^{\mathrm{T}}\in\mathbb{R}^R$。如果对于每一种资源 i，都有 $\boldsymbol{T}^i\leqslant\boldsymbol{S}^i$，则我们说 $\boldsymbol{T}\leqslant\boldsymbol{S}$。例如，如果资源窗口为 $[\boldsymbol{a},\boldsymbol{b}]$，则资源向量可以定义为集合 $\{\boldsymbol{T}\in\mathbb{R}^R:\boldsymbol{a}\leqslant\boldsymbol{T}\leqslant\boldsymbol{b}\}$。

接下来要引入的概念是资源间隔（Resource Interval），也叫资源窗口（Resource Window）。每个节点 $i\in V$ 的资源窗口可以表示为 $[\boldsymbol{a}_i,\boldsymbol{b}_i]$，其中 $\boldsymbol{a}_i,\boldsymbol{b}_i\in\mathbb{R}^R,\boldsymbol{a}_i\leqslant\boldsymbol{b}_i$。资源的消耗总是随着弧 (i,j) 变化的，前文中也有所介绍。我们称为资源扩展函数（Resource Extension Function，REF）。弧 (i,j) 的 REF 是一个向量，即 $\boldsymbol{f}_{ij}=(f_{ij}^r)_{r=1}^R$。REF $f_{ij}^r:\mathbb{R}^R\to\mathbb{R}$ 是依赖于资源向量 $\boldsymbol{T}_i\in\mathbb{R}^R$ 的，也就是说，是取决于从起点 s 到点 i 的路径上的资源的累计消耗量的。因此 $\boldsymbol{f}_{ij}(\boldsymbol{T}_i)\in\mathbb{R}^R$ 可以解释为沿着路径 $(s,\cdots,i,j)$ 所累积的资源消耗。这里以之前介绍的 SPPTW 为例，仅考虑时间资源，则其 REF 为

$$f_{ij}^r(\boldsymbol{T}_i)=T_i^r+t_{ij}^r \tag{14.10}$$

其中，t_{ij}^r 是与弧 (i,j) 相关的常数，在 SPPTW 中，就是弧 (i,j) 上的行驶时间。一般情况下，REF 是资源可分离的，即不同资源之间不存在相互依赖关系。

给定路径 $P=(v_0,v_1,v_2,\cdots,v_p)$，该路径中包含 $p+1$ 个不同的位置 i，即 $i=0,1,2,\cdots,p$。路径 P 是资源可行（Resource-feasible）的条件是：存在资源向量 $\boldsymbol{T}_i\in[\boldsymbol{a}_{v_i},\boldsymbol{b}_{v_i}],\forall i=0,1,2,\cdots,p$，使得 $\boldsymbol{f}_{v_i,v_{i+1}}(\boldsymbol{T}_i)\leqslant\boldsymbol{T}_{i+1},\forall i=0,1,2,\cdots,p$。我们定义 $\mathcal{T}(P)$ 为路径 P 的最后一个节点 v_p 的所有可行资源向量的集合，即

$$\mathcal{T}(P)=\{\boldsymbol{T}_p\in[\boldsymbol{a}_{v_p},\boldsymbol{b}_{v_p}]:\exists\boldsymbol{T}_i\in[\boldsymbol{a}_{v_i},\boldsymbol{b}_{v_i}],\boldsymbol{f}_{v_i,v_{i+1}}(\boldsymbol{T}_i)\leqslant\boldsymbol{T}_{i+1},\forall i=0,1,2,\cdots,p-1\} \tag{14.11}$$

令 $\mathcal{F}(u,v)$ 为从节点 u 到节点 v 的所有资源可行路径的集合。

2. 路径结构约束

路径结构约束（Path-structural Constraint）是关于路径可行性的进一步建模需求，是独立于资源约束的。一般情况下，路径结构约束不能简单地通过删除网络中的一些弧和点来等价地处理。通常情况下，我们考虑的路径结构约束都是基本路径（Elementary Path）。Elementary Path 就是路径中没有环路的路径，换句话说，就是所有被访问的点，至多只被访问了一次。相反地，一个环（cycle）就是一条起点和终点相同的路径，即路径 $(v_0, v_1, v_2, \cdots, v_p)$ 是环，则 $v_0 = v_p, p > 1$。我们将路径长度小于或等于 k 的环称为 k-cycle。

这里用 $\mathcal{G}$ 表示满足路径结构约束的所有可行路径。

对于 Elementary SPPRC（ESPPRC），则 $\mathcal{G} = \{\text{Elementary Path}\}$。在无环图中（Acyclic Graph），所有路径都是基本路径，因此 SPPRC 和 ESPPRC 是相同的。但是对于有环图来讲，ESPPRC 已经被证明是强 NP-hard 问题（Dror，1994），并由 Beasley 和 Christofides 于 1989 年首先研究和解决。在许多 VRP 的应用中，其定价问题就是 ESPPRC。

对于 SPPRC，$\mathcal{G} = \{\text{All Path}\}$，即不考虑路径结构约束。许多车辆和人员排班问题的子问题都是 SPPRC（Desrosiers et al.，1984）。

由于 ESPPRC 很难求解（在某些情况下是非常困难的），在有环图上建模的 VRP 也常常用求解 SPPRC 的方法代替，因为 SPPRC 可以用伪多项式时间的精确算法进行求解。用 SPPRC 代替 ESPPRC 实际上是对 VRP 子问题的一种松弛。将 ESPPRC 松弛成 SPPRC 虽然会使得子问题更容易求解，但是这种松弛不一定总是好的。比如在使用分支定价算法求解 VRP 时，子问题如果松弛成 SPPRC，很可能会导致获得的下界比较松，而且很有可能导致分支树（Branch and Bound tree）变得非常大，反而对求解效率造成不利的影响（Desaulniers et al.，2006）。

除了 ESPPRC 和 SPPRC，还有一些路径结构约束，例如：

（1）k 环消除的 SPPRC（SPPRC with k-cycle elimination，SPPRC-k-cyc）；

（2）带禁止路径的 SPPRC（SPPRC with forbidden paths，SPPRCFP）；

（3）带先后顺序和配对约束的 SPPRC（SPPRC with Precedence constraints and pairing constraints），这一类问题主要针对取货和送货的路径结构约束。

这里我们仅介绍 k 环消除约束的 SPPRC。

对于 k 环消除约束的 SPPRC，则 $\mathcal{G} = \{k\text{-cycle-free path}\}$，即没有 k-cycle 的所有路径的集合。这也是解决 ESPPRC 比较难求解的一个折中方案，即禁止长度较短的环。根据 Solomon VRP 标杆算例的测试结果，仅取 k 比较小的值，也可以比较显著地提升 Master Problem 的下界。这说明，除了求解纯 SPPRC 之外，还需要对 SPPRC 的解进行额外的约束，也就是删除一些环，这相对于直接求解 ESPPRC 来讲会容易很多。Houck et al.，1980 首次讨论了 $k = 2$ 的情况，而这种做法也被之后的研究广泛采用（Kolen et al.，1987；Desrochers et al.，1992）。

对于目标函数的分类，这里不做解释，感兴趣的读者可以阅读文献（Desaulniers et al.，2006）。

3. 基础网络

SPPRC 还可以根据其基础网络（Underlying Network）是无环的还是有环的加以区分。有环的网络意味着在 G 中存在无穷多个不同的路径（不一定是资源可行路径和满足路径结构约束的路径）。因此，SPPRC 可能是无界的（如果网络中存在负环，则 SPPRC 可能是无界的）。这里不考虑无界的情况。

对于一些特殊的情况，可以通过离散化的手段，将有环图转化为无环图，而求解原网络图上的 ESPPRC，就等价于求解离散化之后的网络图上的 SPPRC。如果存在至少一个非递减资源 r （即 $f_{ij}^r - T_i^r > 0$，或者 $t_{ij}^r > 0$），该网络图 $G = (V, A)$ 就有可能被转化成一个无环的时空网络（Acyclic Time-space Network）。对于每个节点 $v \in V$，我们将该点对应的资源 r 的资源窗口（Resource Interval）离散化成 p 段，用一些副本 $\text{copy}^1(v), \text{copy}^2(v), \cdots, \text{copy}^p(v)$ 来代替 G 中的点 v。然而，这种转换只是一种形式手段，转换完成后，仍然可以套用文献（Desaulniers et al., 1998）中提出的方法进行建模。当然，按照这种方法转化之后，求解转化后的无环的时空网络上的 SPPRC，就等价于求原网络图上的 ESPPRC。

14.4.2 标签算法

求解 SPPRC 的动态规划算法又叫标签算法。该算法从一个初始路径 $P = (s)$ 开始，一步一步地沿着所有的可行方向拓展初始路径，从而生成所有的可行路径。该算法的效率取决于识别和舍弃那些对于构建或者生成帕累托最优（Pareto-optimal）路径集合无用的路径的能力。识别无用路径是通过优超准则（Dominance Rule）实现的，该准则非常依赖于路径结构约束和 REF 的特性。

为了方便描述，标签算法中的路径及其资源向量用 label（标签）进行编码或者表示，通常将一个部分路径及其资源向量称为一个标签（label）。

对于给定的路径 $P = (v_0, v_1, v_2, \cdots, v_p)$，我们用 $v(P) = v_p$ 表示路径 P 的最后一个节点。如果路径 P 是路径 $Q = (w_0, w_1, w_2, \cdots, w_q)$ 的一个可行拓展，则有

$$(Q, P) = (w_0, w_1, w_2, \cdots, w_q, v_0, v_1, v_2, \cdots, v_p) \in \mathcal{F}(w_0, v_p) \cup \mathcal{G}$$

Q 的所有可行拓展的集合表示为 $\mathcal{E}(Q) = \{P : (Q, P) \in \mathcal{F}(w_0, v(P)) \cup \mathcal{G}\}$。

标签算法依赖于对两个集合的操作。第一个集合表示为 $\mathcal{U}$，是未处理的路径的集合，$\mathcal{U}$ 中的路径都是未被扩展的。第二个集合 $\mathcal{P}$ 是有用路径的集合。有用的路径 $P \in \mathcal{P}$ 是已经被处理过的。它们已经被确定为是帕累托最优路径或可能是帕累托最优路径的前缀（注意，帕累托最优路径可能会有非帕累托最优的前缀）。集合 $\mathcal{U}$ 和 $\mathcal{P}$ 在标签算法过程中都是动态变化的。最终 $\mathcal{U}$ 变为空时，算法结束，而最优解一定在集合 $\mathcal{P}$ 中，我们只需要将其筛选出来即可。

标签算法中，我们首先初始化集合 $\mathcal{U} = P_0, \mathcal{P} = \varnothing$，其中 $P_0 = (s)$ 是初始路径。每一条路径 $P = (v_0, v_1, v_2, \cdots, v_p) \in \mathcal{F}$ 都是从初始路径 P_0 拓展而来的，即 $(v_1, v_2, \cdots, v_p) \in \mathcal{E}(P_0)$。

在处理完所有拓展后，我们需要执行最后的筛选，也就是从有用路径集合 $\mathcal{P}$ 中，筛选出帕累托最优解 $\mathcal{S}$（有可能有多个）。在每一轮的循环中，拓展步骤结束之后，我们可以对 $\mathcal{U}$ 和 $\mathcal{P}$ 中的标签（也就是路径）进行优超（Dominance）。这一步往往是加速算法的关键。

14.4.3 标签算法的伪代码

上一节我们详细介绍了标签算法的原理，本节我们根据之前的介绍，整理出标签算法的伪代码。

Algorithm 14 SPPRC/ESPPRC 的通用动态规划算法（标签算法）

Input: 网络数据（弧集，距离信息）和客户数据（时间窗），起始点 s, 终止点 t
Output: 最短路 optPath

```
1: 初始化: 设置 U ← {(s)} 和 P ← ∅
2: while U 非空 do
3:     (**** 拓展路径 **** )
4:     选择一条部分路径 Q ∈ U 并将 Q 从 U 中删除
5:     for 弧 (v_Q, w) ∈ A 是从点 v_Q 出去的 do
6:         if (Q, w) 是可行拓展 then
7:             将 (Q, w) 添加至 U
8:         end if
9:     end for
10:    P ← P ∪ {Q}
11:    (****  优超 **** )
12:    if 任意的条件（可自己定义） then
13:        对 U ∪ P 中最后一个节点为 v 的部分路径执行优超算法
14:    end if
15: end while
16: (**** 筛查 **** )
17: optPath ← 对 P 进行筛查，识别出最优解
18: return optPath
```

如果求解的问题是 ESPPRC，则只需将伪代码中第 6 行改为“如果拓展 (Q,w) 是可行的，且满足基本路径约束”。另外，优超准则部分也需要做相应的修改。

针对以上伪代码，有以下几点需要注意的地方。

（1）如果只执行路径扩展步骤，而不执行优超步骤，我们将得到所有的可行路径，即 $\mathcal{P}=\mathcal{F}$。

（2）在路径扩展时，可以根据不同的规则选择下一个被拓展的路径，也就是说，在选择 $Q\in\mathcal{U}$ 时，可以有不同的策略。不同的选择策略、基础网络和 REF 将会使得算法变成标签设定算法或者标签校正算法。

（3）在算法进行过程中，优超算法（Dominance Algorithm）可以随时被调用。为了减少计算量，可以设置优超算法被调用的时机，以便于优超算法能够同时删除多条路径。

（4）对于筛选步骤，是有一些比较有效的算法的，例如从 $\mathcal{P}$ 中识别出帕累托最优路径（Bentley，1980；Kung et al.，1975）。

14.4.4 标签设定算法和标签校正算法

上一节我们提到了标签设定（Label Setting）和标签校正（Label Correction）算法。二者都是动态规划算法，但是又有一些区别。

在标签设定算法中，那些选择要扩展的标签（在路径扩展步骤中）一直保留到标记过程结束。在后续的优超算法调用中，它们将不会被识别为可删除或者可丢弃的。Dijkstra 算法就是一种典型的标签设定算法。不能保证这种行为的标签算法称为标签校正算法。也就是说，被扩展的标签在算法调用中可能会被丢弃。

14.4.5 优超准则和优超算法

优超准则（Dominance Rule）和优超算法（Dominance Algorithm）是提升 SPPRC 和 ESPPRC 求解的重要武器。如果一条路径 Q 既不能产生帕累托最优解 $\mathrm{PO}(v(Q))$，也不能产生可行扩展 $Q' \in \mathcal{E}(Q)$，使得路径 (Q, Q') 可以产生帕累托最优解 $\mathrm{PO}(v(Q'))$，则路径 Q 就可以被删除。如果引入的优超准则和优超算法非常有效，则在迭代过程中 $\mathcal{U}$ 和 $\mathcal{P}$ 的集合中的元素的个数会明显减少，从而可以显著地加快求解速度。

一般来讲，优超准则是通过比较两个具有相同尾节点的路径 P 和 $Q(v(P) = v(Q))$ 的资源向量 $\boldsymbol{T}(P)$、$\boldsymbol{T}(Q)$ 和可行拓展集合 $\mathcal{E}(P)$、$\mathcal{E}(Q)$ 来识别无用路径的。本节来探讨 SPPRC、ESPPRC 和 SPPRC-2-cyc 共 3 种情况下的优超准则。还有一种是 SPPRC-k-cyc，本章不做详细阐述，感兴趣的读者可以参考文献（Desaulniers et al.，2006）。

1. SPPRC

给定两个不同的路径 $P, Q \in \mathcal{U} \cup \mathcal{P}$，$v(P) = v(Q)$，且 $\boldsymbol{T}(P) \leqslant \boldsymbol{T}(Q)$，并且满足下面两种情况，那么我们就可以删除路径 Q。

（1）$\boldsymbol{T}(P) \leqslant \boldsymbol{T}(Q)$ 表示 $\boldsymbol{T}(Q)$ 不可能产生更好的帕累托最优解。

（2）如果通过探索 Q 的所有可行扩展，得出 $\mathcal{E}(Q) \subseteq \mathcal{E}(P)$，则可以删除 Q。特别的，如果没有任何路径结构约束，并且 REF 都是非递减的，则显然有 $\mathcal{E}(Q) \subseteq \mathcal{E}(P)$。

例如，在 SPPTW 中，$\boldsymbol{T}(P) \leqslant \boldsymbol{T}(Q)$ 就表示与 Q 相比，P 所用时间短，花费成本也小。并且由于 SPPTW 中资源全是非递减的，因此就有 $\mathcal{E}(Q) \subseteq \mathcal{E}(P)$。根据上文介绍，我们得知，$Q$ 可以被删除。

2. ESPPRC

对于 ESPPRC 来讲，$\boldsymbol{T}(P) \leqslant \boldsymbol{T}(Q)$ 并不能说明 $\mathcal{E}(Q) \subseteq \mathcal{E}(P)$。原因是，在 ESPPRC 中，路径 $P \in \mathcal{G}$ 只能被拓展到那些还没有被访问的点处。我们用 $V(P)$ 表示已经被访问的

点的集合。因此，ESPPRC 中，删除 Q 的条件为 $\boldsymbol{T}(P) \leqslant \boldsymbol{T}(Q)$，且 $V(P) \subseteq V(Q)$（这两个条件同时满足，就可以说明 $\mathcal{E}(Q) \subseteq \mathcal{E}(P)$）。其他详细解释见文献（Desaulniers et al., 2006）。

3. SPPRC-2-cyc

对于 2-cycle Elimination 的情形，优超准则的一种直观描述如下：只保留一条帕累托最佳路径 P_1 和一条次优路径 P_2，这两条路径是从不同的前序节点（Predecessor node）扩展而来的。对于任意路径 $P=(v_0, v_1, v_2, \cdots, v_p), p \geqslant 1$，节点 v_{p-1} 被称为前序节点，用 $\text{pred}(P)$ 表示。则任给三条路径 P_1, P_2, Q，满足 $v(P_1)=v(P_2)=v(Q)$，且 $\boldsymbol{T}(P_1), \boldsymbol{T}(P_2) \leqslant \boldsymbol{T}(Q), \text{pred}(P_1) \neq \text{pred}(P_2)$，则我们可以删除 Q 而保留 P_1, P_2。原因是 $\text{pred}(P_1) \neq \text{pred}(P_2)$ 表明 $\mathcal{E}(P) \subseteq \mathcal{E}(P_1) \cup \mathcal{E}(P_2)$。

14.4.6 Python 实现标签算法求解 SPPRC

介绍完算法理论细节之后，现在进入实战环节。本节我们给出 Python 实现标签算法求解 SPPRC 的完整代码。我们用 Solomon VRPTW 标杆算例来测试算法。本代码为入门代码，不保证效率，读者需要自行改进。

1. 读取算例数据

读取算例数据代码如下。

readData

```
class Data:
    customerNum = 0;
    nodeNum     = 0;
    vehicleNum  = 0;
    capacity    = 0;
    cor_X       = [];
    cor_Y       = [];
    demand      = [];
    serviceTime = [];
    readyTime   = [];
    dueTime     = [];
    disMatrix   = [[]];# 读取数据

# function to read data from .txt files
def readData(data, path, customerNum):
    data.customerNum = customerNum;
    data.nodeNum = customerNum + 2;
    f = open(path, 'r');
    lines = f.readlines();
    count = 0;
```

```
    # read the info
    for line in lines:
        count = count + 1;
        if(count == 5):
            line = line[:-1].strip();
            str = re.split(r" +", line);
            data.vehicleNum = int(str[0]);
            data.capacity = float(str[1]);
        elif(count >= 10 and count <= 10 + customerNum):
            line = line[:-1];
            str = re.split(r" +", line);
            data.cor_X.append(float(str[2]));
            data.cor_Y.append(float(str[3]));
            data.demand.append(float(str[4]));
            data.readyTime.append(float(str[5]));
            data.dueTime.append(float(str[6]));
            data.serviceTime.append(float(str[7]));

    data.cor_X.append(data.cor_X[0]);
    data.cor_Y.append(data.cor_Y[0]);
    data.demand.append(data.demand[0]);
    data.readyTime.append(data.readyTime[0]);
    data.dueTime.append(data.dueTime[0]);
    data.serviceTime.append(data.serviceTime[0]);


    # compute the distance matrix
    data.disMatrix = [([0] * data.nodeNum) for p in range(data.nodeNum)];
       # 初始化距离矩阵的维度,防止浅拷贝
    # data.disMatrix = [[0] * nodeNum] * nodeNum]; 这个是浅拷贝，容易重复
    for i in range(0, data.nodeNum):
        for j in range(0, data.nodeNum):
            temp = (data.cor_X[i] - data.cor_X[j])**2 + (data.cor_Y[i] -
       data.cor_Y[j])**2;
            data.disMatrix[i][j] = math.sqrt(temp);
            # if(i == j):
            # data.disMatrix[i][j] = 0;
            # print("%6.2f" % (math.sqrt(temp)), end = " ");
            temp = 0;

    return data;
```

```
def printData(data, customerNum):
    print("下面打印数据\n");
    print("vehicle number = %4d" % data.vehicleNum);
    print("vehicle capacity = %4d" % data.capacity);
    for i in range(len(data.demand)):
        print('{0}\t{1}\t{2}\t{3}'.format(data.demand[i], data.readyTime[i],
      data.dueTime[i],  data.serviceTime[i]));

    print("-------距离矩阵-------\n");
    for i in range(data.nodeNum):
        for j in range(data.nodeNum):
            # print("%d   %d" % (i, j));
            print("%6.2f" % (data.disMatrix[i][j]), end = " ");
        print();

# reading data
data = Data()

path = 'Solomn标准VRP算例/solomon-100/In/c101.txt'
customerNum = 100
readData(data, path, customerNum)
printData(data, customerNum)
```

2. 构建网络图

我们用 Python 的 networkx 包来构建网络图，实现数据的可视化。代码如下。

buildGraph

```
# 构建有向图对象
Graph = nx.DiGraph()
cnt = 0
pos_location = {}
nodes_col = {}
for i in range(data.nodeNum):
    X_coor = data.cor_X[i]
    Y_coor = data.cor_Y[i]
    name = str(i)
    nodes_col[name] = 'gray'
    node_type = 'customer'
    if(i == 0):
        node_type = 'depot'
    Graph.add_node(name
                , ID = i
```

```
                , node_type = node_type
                , time_window = (data.readyTime[i], data.dueTime[i])
                , arrive_time = 10000    # 这个是时间标签1
                , demand = data.demand
                , serviceTime = data.serviceTime
                , x_coor = X_coor
                , y_coor = Y_coor
                , min_dis = 0            # 这个是距离标签2
                , previous_node = None   # 这个是前序节点标签3
                )

    pos_location[name] = (X_coor, Y_coor)
# add edges into the graph
for i in range(data.nodeNum):
    for j in range(data.nodeNum):
        if(i == j or (i == 0 and j == data.nodeNum - 1) or (j == 0 and i ==
       data.nodeNum - 1)):
            pass
        else:
            Graph.add_edge(str(i), str(j)
                           , travelTime = data.disMatrix[i][j]
                           , length = data.disMatrix[i][j]
                          )

# 画出图
# plt.rcParams['figure.figsize'] = (0.6 * trip_num, trip_num) # 单位是inch
nodes_col['0'] = 'red'
nodes_col[str(data.nodeNum-1)] = 'red'
plt.rcParams['figure.figsize'] = (10, 10) # 单位是inch
nx.draw(Graph
        , pos=pos_location
        , with_labels = True
        , node_size = 200
        , node_color = nodes_col.values()    #'y'
        , font_size = 15
        , font_family = 'arial'
        , edge_color = 'grey'    #'grey'  # b, k, m, g,
       )
fig_name = 'network_' + str(customerNum) + '.jpg'
plt.savefig(fig_name, dpi=600)
plt.show()
```

网络图如图 14.11 所示。

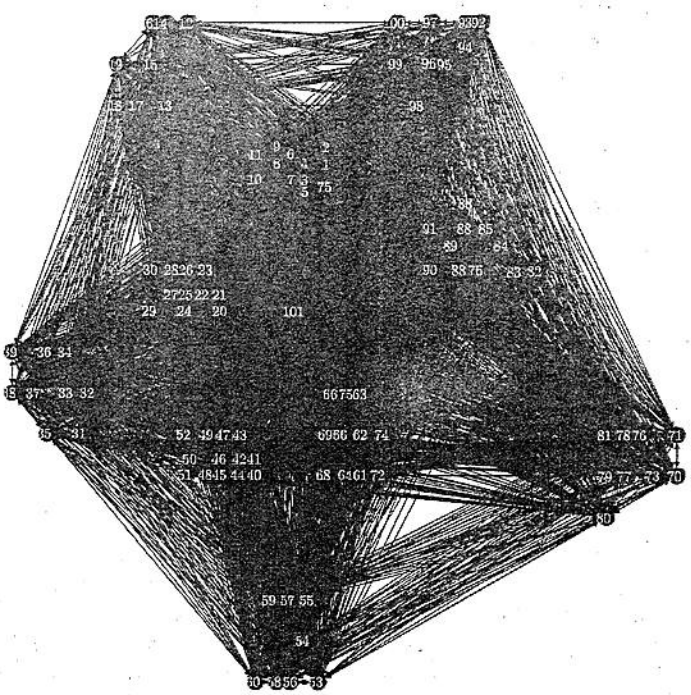


图 14.11　网络图：100 个顾客

3. 标签算法

标签算法代码如下。

Labelling Algorithm

```
class Label:
    path = []
    time = 0
    dis = 0


def labelling_SPPRC(Graph, org, des):
    # initialize Queue
    Queue = []
    # creat initial label
    label = Label()
    label.path = [org]
    label.dis = 0
    label.time = 0
```

```
    Queue.append(label)
    Paths = {}
    cnt = 0
    while(len(Queue) > 0):
        cnt += 1
        current_path = Queue[0]
        Queue.remove(current_path)
        # extend the label
        last_node = current_path.path[-1]
        for child in Graph.successors(last_node):
            extended_path = copy.deepcopy(current_path)
            arc_key = (last_node, child)
            # justify whether the extension is feasible
            arrive_time = current_path.time + Graph.edges[arc_key]['
travelTime']
            time_window = Graph.nodes[child]['time_window']
            if((child not in extended_path.path) and arrive_time >= time_
window[0] and arrive_time <= time_window[1]):
                # the extension is feasible
                # print('extension is feasible', 'arc :', arc_key)
                extended_path.path.append(child)
                extended_path.dis += Graph.edges[arc_key]['length']
                extended_path.time += Graph.edges[arc_key]['travelTime']
                Queue.append(extended_path)
                print('extended_path : ', extended_path.path)
            else:
                pass
                # print('extension is infeasible', 'arc :', arc_key)
        Paths[cnt] = current_path

        # dominate step
        '''
        Add dominate rule function
        '''
        Queue, Paths = dominate(Queue, Paths)

    # filtering Paths, only keep solutions from org to des, delete other
    # paths
    PathsCopy = copy.deepcopy(Paths)
    for key in PathsCopy.keys():
        if(Paths[key].path[-1] != des):
            Paths.pop(key)
```

```
54
55      # choose optimal solution
56      opt_path = {}
57      min_distance = 10000000
58      for key in Paths.keys():
59          if(Paths[key].dis < min_distance):
60              min_distance = Paths[key].dis
61              opt_path['1'] = Paths[key]
62
63      return Graph, Queue, Paths, PathsCopy, opt_path
```

4. 优超准则

我们加入优超准则（Dominate rule）。若路径 P,Q 满足

$$t(P) \leqslant t(Q) \tag{14.12}$$

$$c(P) \leqslant c(Q) \tag{14.13}$$

$$v(P) = v(Q) \tag{14.14}$$

则 Q 被删除。其中，$t(P)$ 和 $c(P)$ 分别表示路径 P 的时间和成本，$v(P)$ 和 $v(Q)$ 分别表示路径 P 和 Q 访问的最后一个节点。优超准则的具体实现函数如下：

dominate rule

```
1  def dominate(Queue, Paths):
2      QueueCopy = copy.deepcopy(Queue)
3      PathsCopy = copy.deepcopy(Paths)
4
5      # dominate Queue
6      for label in QueueCopy:
7          for another_label in Queue:
8              if(label.path[-1] == another_label.path[-1] and label.time <
         another_label.time and label.dis < another_label.dis):
9                  Queue.remove(another_label)
10                 print('dominated path (Q) : ', another_label.path)
11
12     # dominate Paths
13     for key_1 in PathsCopy.keys():
14         for key_2 in PathsCopy.keys():
15             if(PathsCopy[key_1].path[-1] == PathsCopy[key_2].path[-1]
16                and PathsCopy[key_1].time < PathsCopy[key_2].time
17                and PathsCopy[key_1].dis < PathsCopy[key_2].dis
18                and (key_2 in Paths.keys())):
```

```
                Paths.pop(key_2)
                print('dominated path (P) : ', PathsCopy[key_1].path)

    return Queue, Paths
```

5. 算例测试

取算例 C101 中的前 10 个点作为测试数据对标签算法进行测试。

Test Labelling Algorithm

```
org = '0'
des = str(data.nodeNum - 1)
Graph, Queue, Paths, PathsCopy, opt_path = labelling_SPPRC(Graph, org, des)

for key in Paths.keys():
    print(Paths[key].path)

print('optimal path : ', opt_path['1'].path )
print('optimal path (distance): ', opt_path['1'].dis)
print('optimal path (time): ', opt_path['1'].time)
```

最终结果如下。

Result of Labelling Algorithm

```
optimal path :  ['0', '20', '101']
optimal path (distance):  20.0
optimal path (time):  20.0
```

运行时间为 5 秒。

接着测试 600 个点的大算例。用 c600 为例，结果如下。

Result of 600 customers instance

```
optimal path :  ['0', '59', '13', '173', '90', '86', '490', '391', '572', '
      322', '267', '492', '551', '599']
optimal path (distance):  512.0103739069432
optimal path (time):  512.0103739069432

running time : 20min 12s
```

接着测试 1000 个点的最大算例。用 c1000 为例，结果如下。

Result of 1000 customers instance

```
optimal path :  ['0', '183', '1001', '997', '836', '638', '155', '87', '69',
        '1', '755', '976', '70', '704', '458', '629', '107', '99', '798', '
       829', '289', '51', '515', '649', '591', '1000']
optimal path (distance):  827.3615049912353
optimal path (time):  827.3615049912353

Running Time: 1h 31min 33s
```

14.5 Python 实现标签算法结合列生成求解 VRPTW

在第 13 章中介绍了用列生成算法求解 VRPTW。我们用列生成算法的思想，将 VRPTW 分解成为主问题和子问题，其中，子问题是一个 ESPPRC。在第 13 章，我们直接调用求解器对 ESPPRC 进行了求解，但是由于 ESPPRC 也是 NP-hard 问题，所以求解效果并不理想。本节我们尝试用前文介绍过的标签算法来加速子问题的求解。当然我们也可以直接将子问题松弛成 SPPRC，也就是去掉每条路径中节点至多只能被访问一次的约束。执行松弛操作的目的就是为了加速子问题求解，但是同时可能会导致主问题得到的下界非常差。关于这点，我们不做详细讨论。为了进一步加速整个求解过程，我们采用一种常见的启发式算法——最近邻居（Nearest Neighbor）算法来生成 RMP 的初始可行列。

14.5.1 初始化 RMP

我们用最近邻居算法生成初始可行列，其伪代码如下。其主要思想就是首先创建一条初始路径 $[0,0]$，然后不断地寻找最近的节点，插入当前的路径中，直到违背载重约束为止，就生成了一辆车的路径。重复该过程，直到所有的客户都被访问。

Algorithm 15 最近邻居算法

Input: 给定: 图 $G=(V,E)$, vehicle_num, customer_num, 车场点 O, 车容量 Q
Output: R

(***** 初始化 *****)
客户集合 $C \leftarrow \{1,2,\cdots,$ customer_num$\}$ 和 $t_c \leftarrow 0$
路径集合 $R \leftarrow \varnothing$
while C 非空 **do**
    单条路径 $P \leftarrow [0,0]$
    (***** 插入步骤 *****)
    while P. load $< Q$ 且 C 非空 **do**
        找到最佳插入客户 v 和对应的插入位置 pos
        v, pos,arrive_time $\leftarrow$ Insertion Position(G, C, P, t_c)
        if P.load + v.demand $> Q$ 且 pos 为空 **then**

```
            break
        else
            P. insert (v, pos)
            C. remove (v)
            更新 t_c ← arrive_time + inserted_customer. service_time
            更新车辆载重 P. load ← P. load + v. demand
        end if
    end while
    R ← R ∪ P
end while
return R
```

其中，Insertion Position(G, C, P) 是为了找到最佳的插入位置和最佳的插入顾客。其输入参数为网络图 G、未被访问的顾客集合 C 和当前路径 P。其伪代码如下。

Algorithm 16 插入位置（Insertion Position）选择算法

Input: 图 $G = (V, E)$, 候选顾客集合 C, 路径 P（将要执行插入操作），当前时间 t_c

Output: inserted_customer, inserted_position, arrive_time

```
(***** 初始化 ***** )
inserted_position ← NULL
inserted_customer ← NULL
最近距离 d_min ← ∞
if len(P) ⩽ 2 then
    inserted_customer ← 距离车场最远的客户点 (需要满足时间窗约束)
    inserted_position ← 1
    arrive_time ← max{t_c + t_{0,inserted_customer}, a_inserted_customer}
    return inserted_customer, inserted_position, arrive_time
end if
for 顾客 p ∈ C do
    v ← 路径 P 的最后一个客户点
    if d_{v,p} < d_min 并且满足时间窗约束 then
        d_min ← d_{v,p}
        inserted_customer ← p
        inserted_position ← len(P) −1
        arrive_time ← max{t_c + t_{0,inserted_customer}, a_inserted_customer}
    end if
end for
return inserted_customer, inserted_position, arrive_time
```

利用最近邻居算法可以非常快地找到一个较好的可行解，作为初始列加入 RMP 中。

14.5.2 标签算法求解子问题

VRPTW 的子问题是一个 ESPPRC，我们利用前文介绍的标签算法求解该问题。在列生成算法的每一步迭代中，我们首先通过求解 RLMP，得到 RLMP 所有约束的对偶变量 λ_i，然后利用 λ_i 更新子问题（ESPPRC）的目标函数，即

$$\min \quad \sum_{i \in N} \sum_{j \in N} (c_{ij} - \lambda_i) x_{ij} \tag{14.15}$$

求解子问题得到的解，就构成新列，加入 RLMP 中，重复此过程，直到没有 Reduced Cost 为负的列产生为止。

初始列的生成也可以采用其他启发式算法，例如 Solomon 提出的 I1 和 I2 算法（Solomon and Marius，1987），Saving Heuristic（节约算法）等（Clarke and Wright，1964）。

第15章　分支定价算法

之前我们介绍了列生成算法，这是一种强大的求解大规模整数规划的算法。但是，我们也指出了列生成算法的缺陷。就是我们在生成新列的过程中，暂时将 RMP 松弛成 LP（即 RLMP），但是在最终形式的 RMP 中，我们将所有变量均设置成 0-1 变量，然后进行求解。我们提到，求解整数规划版本的 RMP 得到的整数解，是原问题最优解的一个上界，但并不一定是原问题的最优解。因此仅仅用列生成算法，并不能保证得到原问题的全局最优解。不过在下面这种情况下，是可以确定列生成算法的解同时也是全局最优解的，那就是当我们将最终形式 RMP 的整数约束去掉，将其松弛成 RLMP，如果 RLMP 的最优解同时也是整数解，则此时列生成算法得到的解就是全局最优解。

遗憾的是，最终形式的 RMP 的线性松弛 RLMP 不总是存在整数最优解。那么，我们不禁要问，如何能够保证总是得到全局最优解呢？一个比较好的解决方法就是将列生成算法与分支定界算法嵌套在一起使用。当最终 RLMP 的解是小数时，我们对取值为小数的变量进行分支，然后在 BB tree 的每个叶子节点处，在执行了分支操作的基础上，继续执行列生成算法，得到新的最终 RMP，紧接着，再次求解这个新的 RMP 对应的 RLMP。我们不断地分支、更新上界和下界，直到算法结束，最终得到的解一定是原问题的最优解。上面的思想，正是著名的分支定价算法（Branch and Price Algorithm）。本章就来详细介绍该算法。

15.1　分支定价算法基本原理概述

分支定价算法就是将列生成算法和分支定界算法嵌套在一起使用，共同配合，从而得到原问题的最优解的一种强大的算法。因此，分支定价算法可以简单理解为列生成算法 + 分支定界算法。

接下来我们简要介绍分支定价算法的原理，然后以 VRPTW 为例，详细介绍分支定价算法的完整流程。

我们知道，列生成算法的目的是找出那些具有负检验数的列（这些列相当于线性规划的单纯形法迭代过程中可以进基的基变量），并动态地将这些列添加到 RMP 中。可惜的是，该过程并不能保证没有任何漏列的情况，即，完备主问题的最优基中基变量对应的列有一部分没有被加进 RMP。这也是为什么最基本的列生成算法很多时候不能保证最优性，只能得到一个比较接近最优解的解的原因。在列生成算法一章中，我们也给了两种更为理论的解释。

第 1 种解释。假定所有列的集合为 Ω，RMP 中的列的集合为 Ω'，且主问题（MP）的解必须是整数，则 MP(Ω) 与 RMP(Ω') 的最优解相同的条件是：MP(Ω) 的最优解被包含在 RMP(Ω') 的所有整数可行解构成的凸包 Conv(Ω') 中。但是如果上述条件不成立，即 Ω' 对应的列构成的模型的整数可行解构成的凸包没有囊括 MP(Ω) 的最优解，则 RMP(Ω') 的解就不是全局最优解。其直接原因就是有一些最优解中被选中的列没有被加进 RMP。

第 2 种解释。全局最优解 z^* 的下界为 $\mathrm{LB} = z_{\mathrm{RLMP}} + \sum_i \tilde{c}_i^*$，其中 $\tilde{c}_i^*$ 为第 i 个子问题的目标函数值；上界为整数规划 RMP 的最优解，即 $\mathrm{UB} = z_{\mathrm{RMP}}$。如果列生成算法结束后，对应的最终的 RLMP 的最优解不是整数解，则有 $z_{\mathrm{RLMP}} < z_{\mathrm{RMP}}$，而由于列生成算法已经结束，因此 $\tilde{c}_i^* \approx 0$，所以 LB < UB，这说明当前目标值 z_{RMP} 不一定是全局最优值。

根据第 1 种解释，要找到最优解的关键就在于，如何找到那些在列生成过程中的漏网之鱼（即被遗漏的最优基中的基变量），并加进 RMP 中。我们的做法仍旧是用列生成算法去找，但是基于当前的最终 RMP，是无法再找到新的具有负检验数的列的。不过，我们可以将目前的最终 RMP 中的“捣蛋鬼”识别出来，将其剔除，形成新的最终 RMP，然后接着用列生成算法，基于新的最终 RMP，再生成若干新列。这些“捣蛋鬼”指的就是当前 RMP 的线性松弛的最优解中取值为小数的列。这里有一点非常关键，就是这些取值为小数的列就一定不是完备主问题最优基中的基变量吗？答案是不一定。也就是取值为小数的列也有可能会出现在全局最优整数解中，只是因为我们没有完整地将最优基中的基变量生成出来，导致这部分最优基变量与非最优基变量组成的 RMP 的整数可行解的凸包没有包含 MP(Ω) 的最优解，所以这部分最优基变量取值为小数。

上述描述中，我们提到，我们可以将“捣蛋鬼”从当前 RMP 中剔除掉，然后继续用列生成算法生成若干新列。那么我们又要问，如此操作之后，新生成的最终 RMP 一定能保证得到最优整数解吗？答案依然是不能。原因也在上文中提到了，就是当前 RMP 中取值为小数的列，同样有可能是 MP(Ω) 的最优基中的基变量，我们将其删去并且禁止之后的列生成过程中不能再生成该列，一旦禁止的这一列正好是 MP(Ω) 最优基中的基变量，那么最终得到的 RMP 的整数可行解的凸包就不能包含 MP(Ω) 的最优整数解，所以，按照这种做法最终不会得到最优解。因此，为了保证最优性，我们还需要进行另外一种互补情况的讨论，即，在另一种情况下，我们规定取值为小数且被上一种情况禁止的列一定包含在该种情况下的最终 RMP 中。以上两种情况就包含了所有的可能，也就保证了没有排除任何整数可行解。当然，在上述两种情况的每种情况下，还有可能再次产生小数解，此时我们可以用相同的方法再执行一次。我们循环上述过程，直到遍历完成所有的可能，就可以得到整数最优解。上述过程实际上就是分支定界算法的思想，每次产生小数解之后，我们分两种情况讨论，就是执行分支操作。在每个节点处，我们首先基于父节点的 RMP 做分支操作，然后接着执行列生成算法的操作，生成一些额外的列。经过不断地更新上界、下界和分支，我们最终可以得到原问题的最优解。这就是分支定价算法的原理。

这里需要特别说明的是，上面的描述中提到的对列进行分支的方法，一般会导致 BB

tree 极不平衡，降低求解效率，因此实际科研中，我们一般采取更巧妙的办法进行分支。

为了方便介绍，我们还是以 VRPTW 为例来详细介绍分支定价算法。注意，本章模型忽略了车辆数限制，在实际问题中，读者可将其加入到模型中。

15.2 分支定价算法求解 VRPTW

15.2.1 VRPTW 的通用列生成建模方法

本节中，我们采取基于路径的建模方式对 VRPTW 进行建模。模型中的每一列都对应一条可行路径。我们用 $C=\{1,2,\cdots,n\}$ 表示所有顾客的集合。假设所有的车都是同质的，即容量等参数均相同。用 R 表示所有的可行路径的集合，也就是说 R 中的所有路径都是满足时间窗约束和车辆的载重约束的。这里需要特别说明，R 中的元素可能非常多。如果每个节点的时间窗都非常大，那么 R 中的元素个数会随着顾客数量的增大呈现阶乘级别的增大。对于可行路径集合中的任意一条可行路径 $r\in R$，我们引入下面的 0-1 变量：

$$y_r=\begin{cases}1, & \text{如果路径 } r \text{ 在最优解中被选中}\\ 0, & \text{其他}\end{cases} \tag{15.1}$$

其中，$r=(0,i_1,i_2,\cdots,0)$ 是一条完整的从 depot（点 0）出发，最终回到 depot 的闭合路径。并且，路径 $r\in R$ 的成本（长度）为 c_r。那么 VRPTW 的基于路径的模型则可以写成下面的集分割问题（Set Partitioning Problem）：

$$\min \quad \sum_{r\in R} c_r y_r \tag{15.2}$$

$$\text{s.t.} \quad \sum_{r\in R} \theta_{ir} y_r = 1, \qquad \forall i\in C \tag{15.3}$$

$$y_r\in\{0,1\}, \qquad \forall r\in R \tag{15.4}$$

其中，θ_{ir} 是一个参数，该参数可以根据每一条路径 $r\in R$ 的具体信息获得。即，如果顾客 $i\in C$ 在路径 r 中被访问到了，那么 $\theta_{ir}=1$，否则 $\theta_{ir}=0$。

上述建模方法非常简洁，但是当所有可行路径集合 R 中的元素数量非常庞大的时候，我们很难直接穷举出完整的 R。一个可行的做法是，先列出一些初始的可行路径，然后再使用列生成算法的思想，用循环的方式，动态地寻找新的可行路径，并将其添加到 RMP 和 R 中。

根据上述思路，可以将 VRPTW 分解成为一个主问题和一个子问题（定价问题）。主问题的模型是基于当前已经列出的可行路径集合 R' 构造出来的，且 $R'\subset R$。我们在之前的章节提到过，由于该主问题仅包含了一部分可行路径，因此又称作限制性主问题（Restricted Master Problem，RMP）。RMP 的模型可以写成

$$\min \quad \sum_{r\in R'} c_r y_r \tag{15.5}$$

$$\text{s.t.} \quad \sum_{r\in R'} \theta_{ir} y_r = 1, \qquad \forall i \in C \tag{15.6}$$

$$y_r \in \{0,1\}, \qquad \forall r \in R' \tag{15.7}$$

为了给子问题提供对偶变量的取值信息，需要将 RMP 松弛成线性规划，也就是将约束 (15.7) 松弛成下面的形式：

$$0 \leqslant y_r \leqslant 1, \qquad \forall r \in R' \tag{15.8}$$

事实上，该约束还可以进一步松弛为

$$y_r \geqslant 0, \qquad \forall r \in R' \tag{15.9}$$

从 (15.8) 到 (15.9) 的松弛是等价的转化，具体解释见文献（Feillet，2010）。为方便读者理解，本文中依然采用 (15.8) 的形式。根据上文的介绍，RMP 的线性松弛可以写成

$$\min \quad \sum_{r\in R'} c_r y_r \tag{15.10}$$

$$\text{s.t.} \quad \sum_{r\in R'} \theta_{ir} y_r = 1, \qquad \forall i \in C \tag{15.11}$$

$$0 \leqslant y_r \leqslant 1, \qquad \forall r \in R' \tag{15.12}$$

求解 (15.10)，我们可以得到约束 (15.11) 对应的 $|C|$ 个对偶变量 $\pi_i, \forall i \in C$。

VRPTW 的子问题就是寻找一条检验数为最小且为负的列，即子问题的目标函数为

$$\min_{r\in R} \quad c_r - \sum_{i\in C} \pi_i \theta_{ir} \tag{15.13}$$

注意，这里是在所有可行路径 R 中去寻找检验数最小的路径。

所有可行路径的集合 R 其实是由一系列约束的可行域描述的，更具体地说，就是由子问题的模型描述的。该子问题就是一个带资源约束的基本最短路问题（Elementary Shortest Path Problem with Resource Constraints，ESPPRC）。根据之前列生成算法章节中的描述，可以将上面的子问题（ESPPRC）的模型写成

$$\min \quad \sum_{i\in N}\sum_{j\in N} (c_{ij} - \pi_i) x_{ij} \tag{15.14}$$

$$\text{s.t.} \quad \sum_{i\in C} d_i \sum_{j\in N} x_{ij} \leqslant q \tag{15.15}$$

$$\sum_{j\in N} x_{0j} = 1 \tag{15.16}$$

$$\sum_{i\in N} x_{ih} - \sum_{j\in N} x_{hj} = 0, \qquad \forall h \in C \tag{15.17}$$

$$\sum_{i\in N} x_{i,n+1} = 1 \tag{15.18}$$